

Modelos Lineares II

Adicionando mais variáveis, transformações e interações

Linear regression is flexible!

Some simple transformations

- The linear model we are using consists of making the parameters of probability distributions change according to some function
- The simplest function is a linear function
- Sometimes the relation between parameters and predictors is not linear
- We can use whatever functional shape we like, but it is useful to use transformations to linearize the relation

$$y_i \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$



$$\log(y_i) \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$

Log transforming the response

Multiplicative increments

- The **logarithmic transformation** of the response is a common transformation when the effect of the predictor on the response is thought to be multiplicative
 - A change of a unit in x is associated with a constant **percentage** change in y
- Many processes benefit from log transformation:
 - Growth is proportional do previous size
 - Any multiplicative process

$$y_i \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$



$$\Delta y \approx \beta \Delta x$$

$$\log(y_i) \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$



$$\% \Delta y \approx (100 \times \beta) \Delta x$$

Biomass by diameter

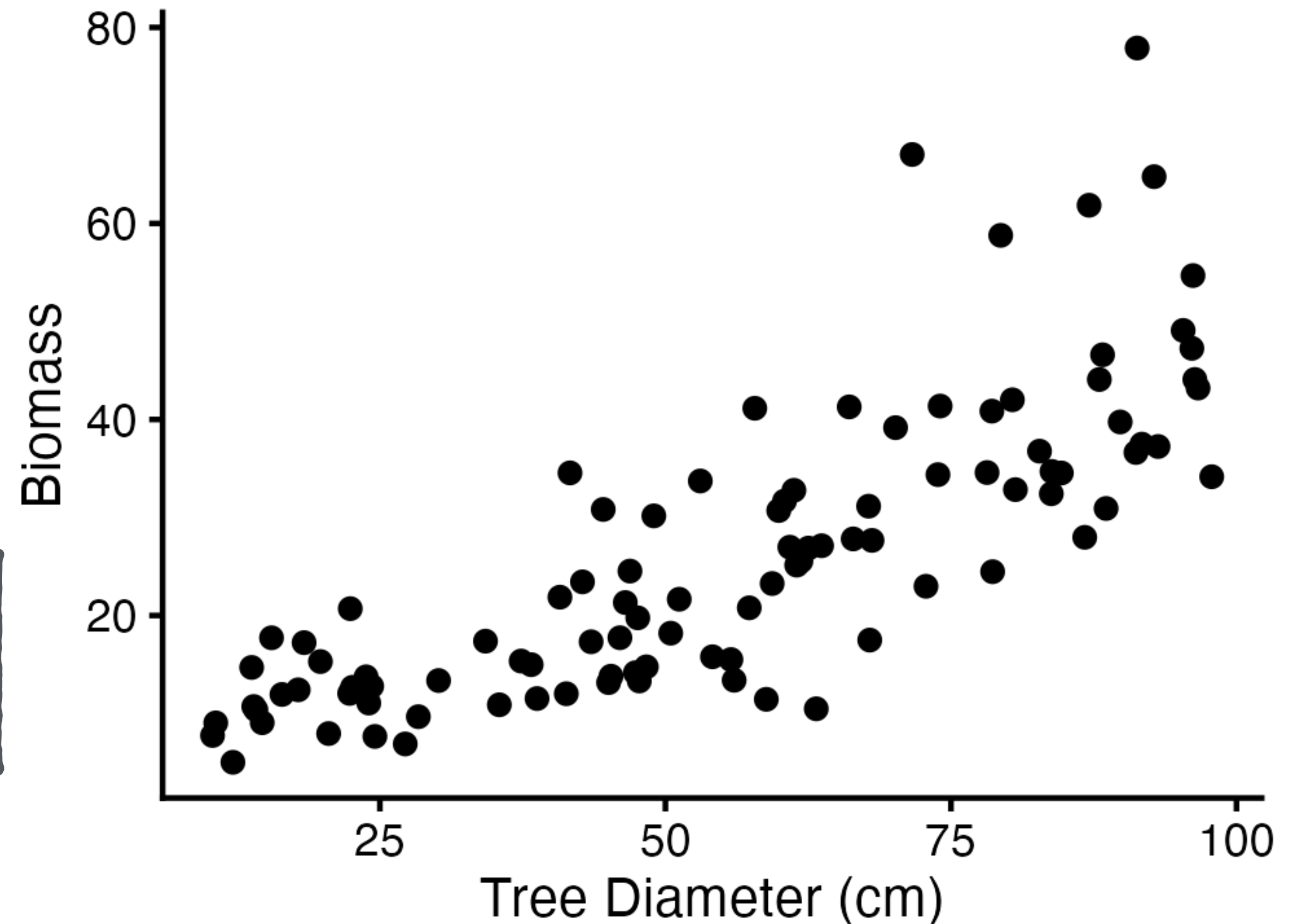
Example of non-linear relation

Option 1: log-transform y

```
df = data.frame(diameter, biomass)
fit = lm(log(biomass) ~ diameter,
         data = df)
```

Option A: Assign y a log-normal likelihood

```
rt_fit = quap(alist(
  biomass ~ lognormal(mu, sigma),
  mu <- a + b*diameter),
  data = df)
```



Biomass by diameter

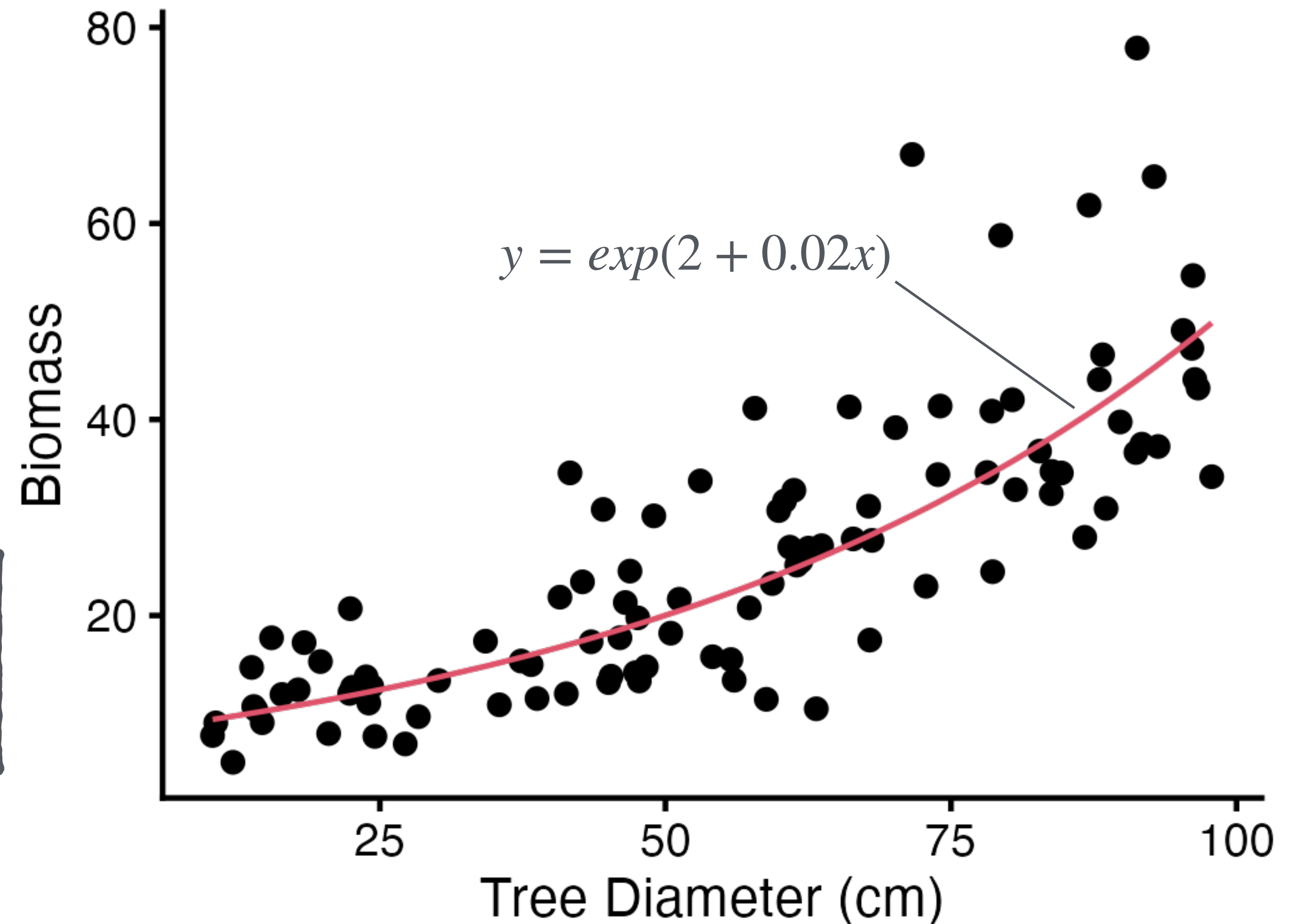
Example of non-linear relation

Option 1: log-transform y

```
df = data.frame(diameter, biomass)
fit = lm(log(biomass) ~ diameter,
         data = df)
```

Option A: Assign y a log-normal likelihood

```
rt_fit = quap(alist(
  biomass ~ lognormal(mu, sigma),
  mu <- a + b*diameter),
  data = df)
```



Biomass by diameter

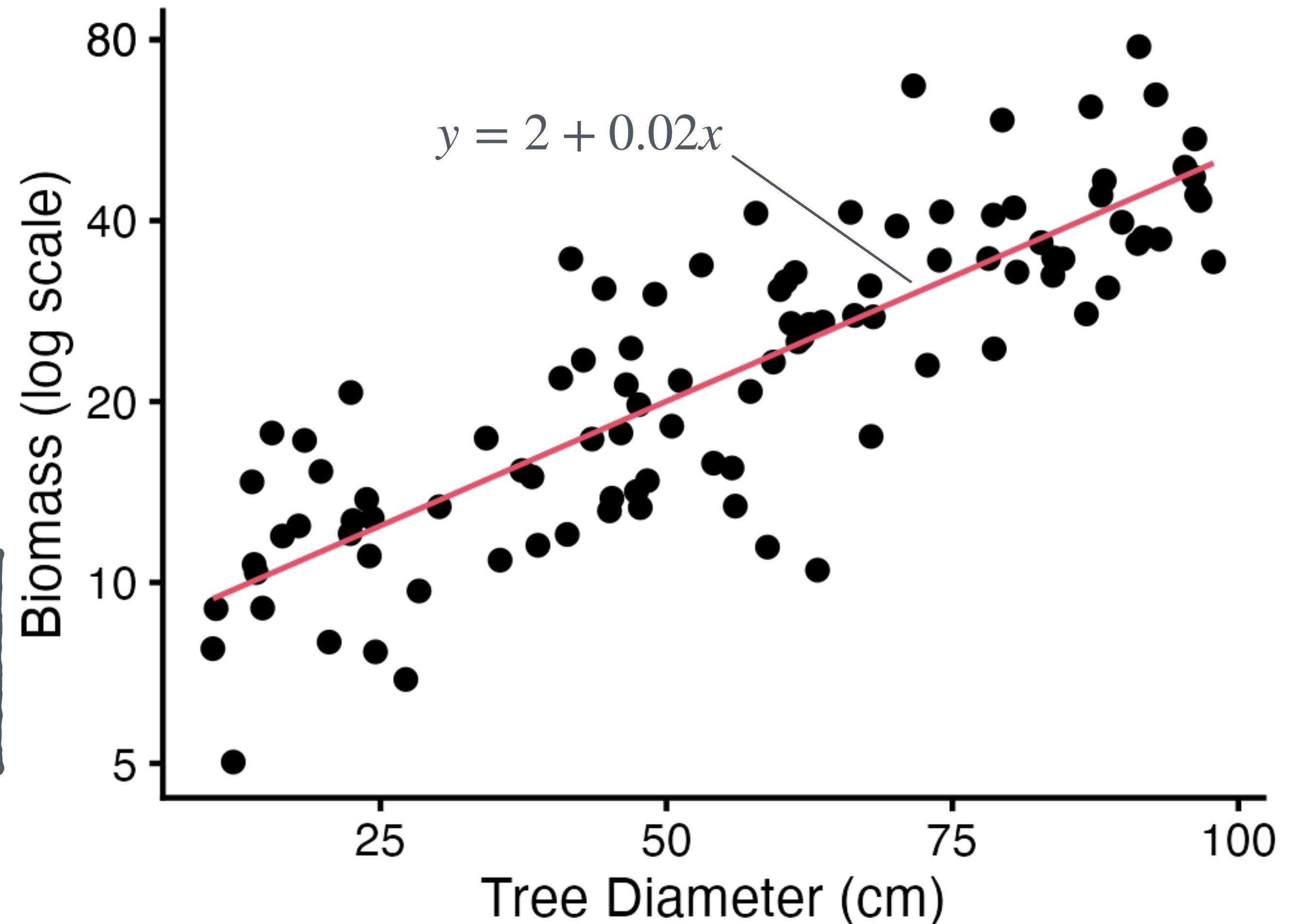
Example of non-linear relation

Option 1: log-transform y

```
df = data.frame(diameter, biomass)
fit = lm(log(biomass) ~ diameter,
         data = df)
```

Option A: Assign y a log-normal likelihood

```
rt_fit = quap(alist(
  biomass ~ lognormal(mu, sigma),
  mu <- a + b*diameter),
  data = df)
```



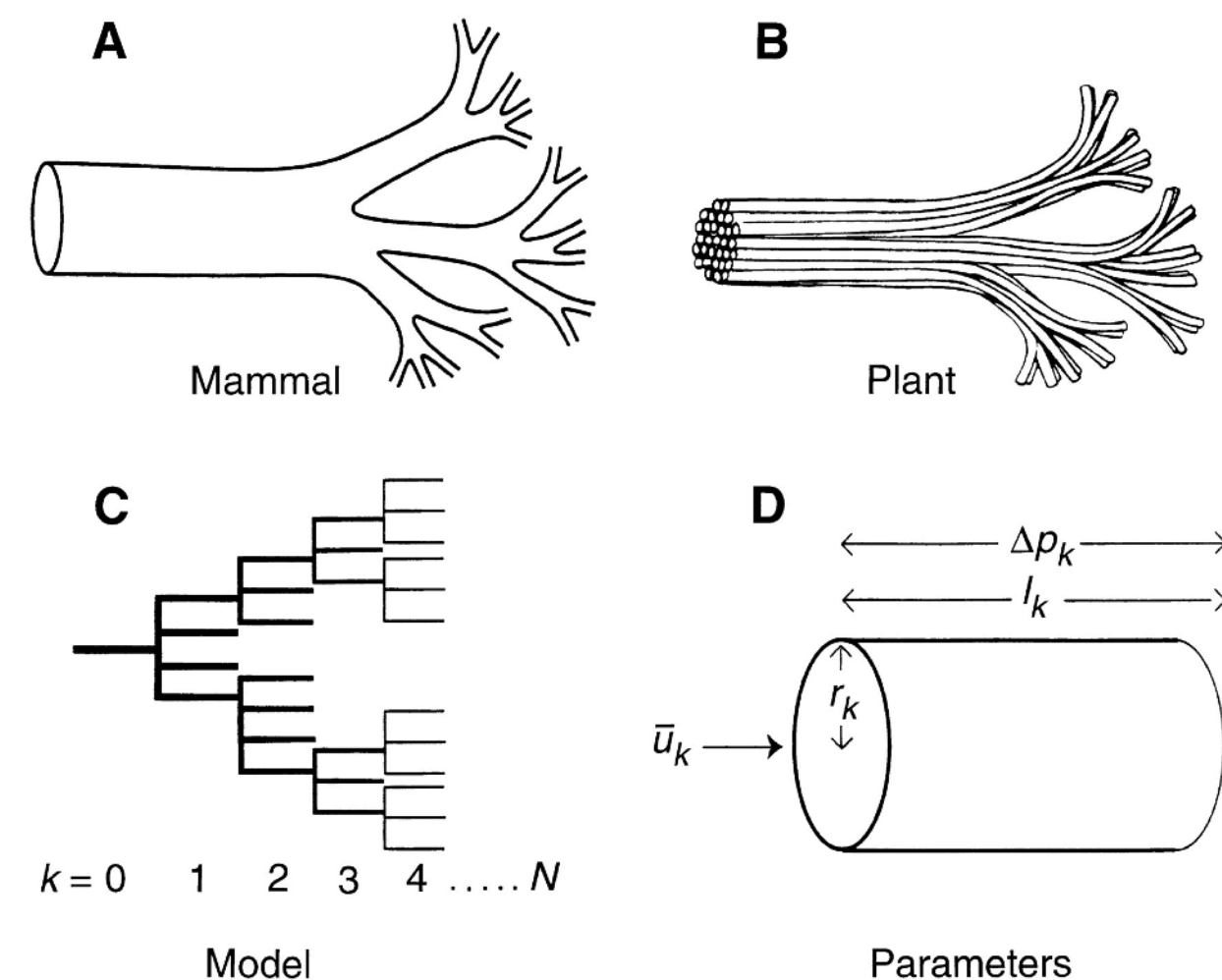
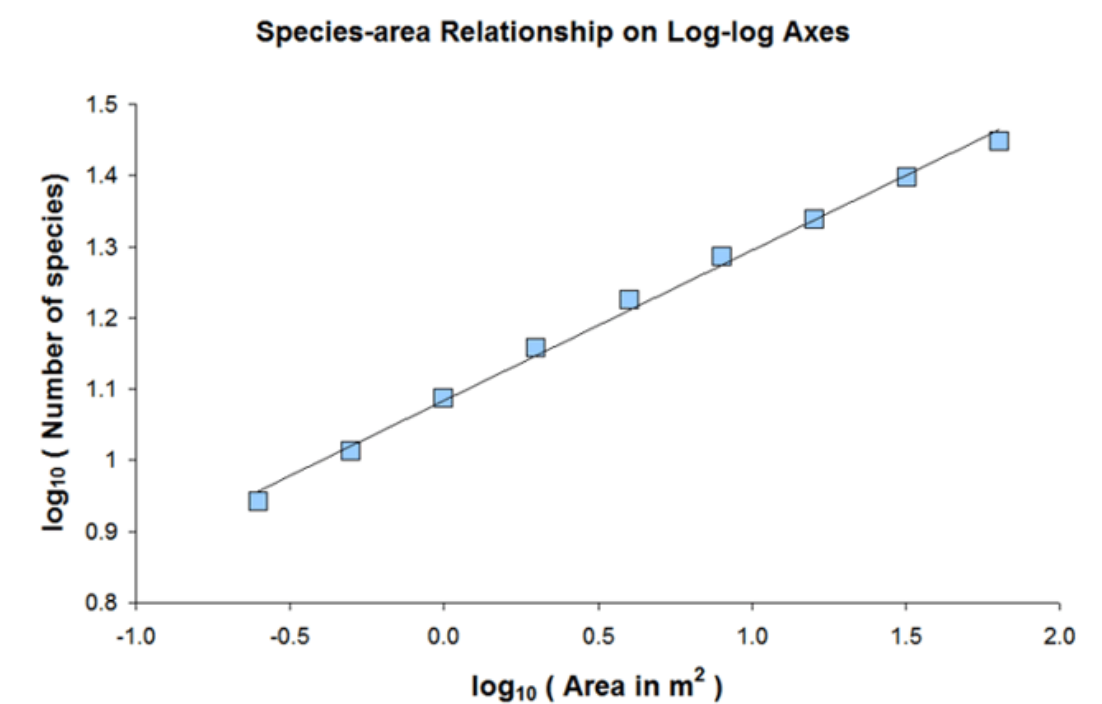
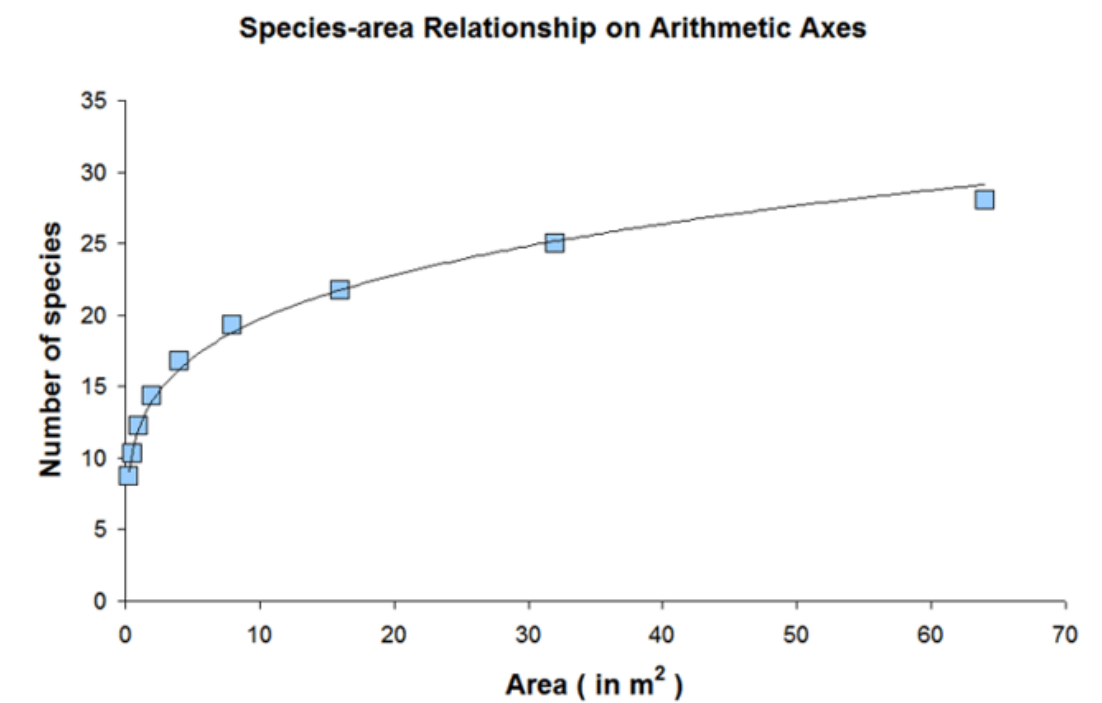
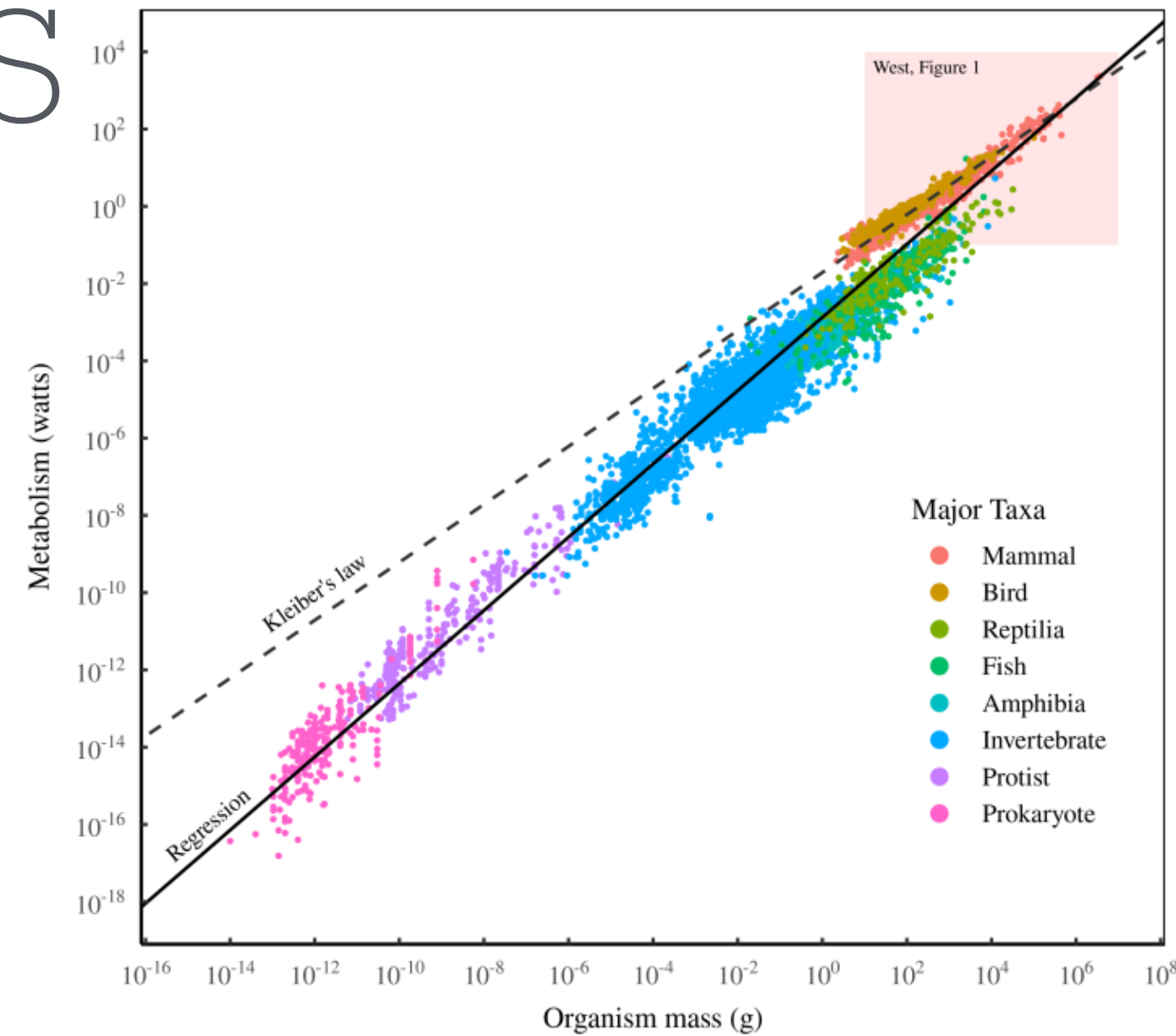
Power-law relations

Log-log regressions

- Several biological relations take the form of power-law relations

$$y \propto ax^b$$

- These appear for different reasons:
 - West et al. (1997) attempt to link scaling laws to fractal relations at different scales
 - Every time a proportional increase leads to a consistent proportional change (like in species-area relations)



Power-law relations

Log-log regressions

- Several biological relations take the form of power-law relations
- We can linearize these relations using a log-log transformation

$$y \propto ax^b$$



Take the log on both sides

$$\log(y) \propto \log(ax^b) = \log(a) + \log(x^b) =$$

$$\log(y) \propto \log(a) + b \log(x)$$

Power-law relations

Log-log regressions

- Several biological relations take the form of power-law relations
- We can linearize these relations using a log-log transformation
- In this model, the slope is an estimate of the exponent of the power-law
- The interpretation of the slope is that a 1% increase in x leads to a β % increase in y

$$\log(y_i) \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta \log(x_i)$$



$$\% \Delta y \approx \beta \% \Delta x$$

Quick reference for log transformations

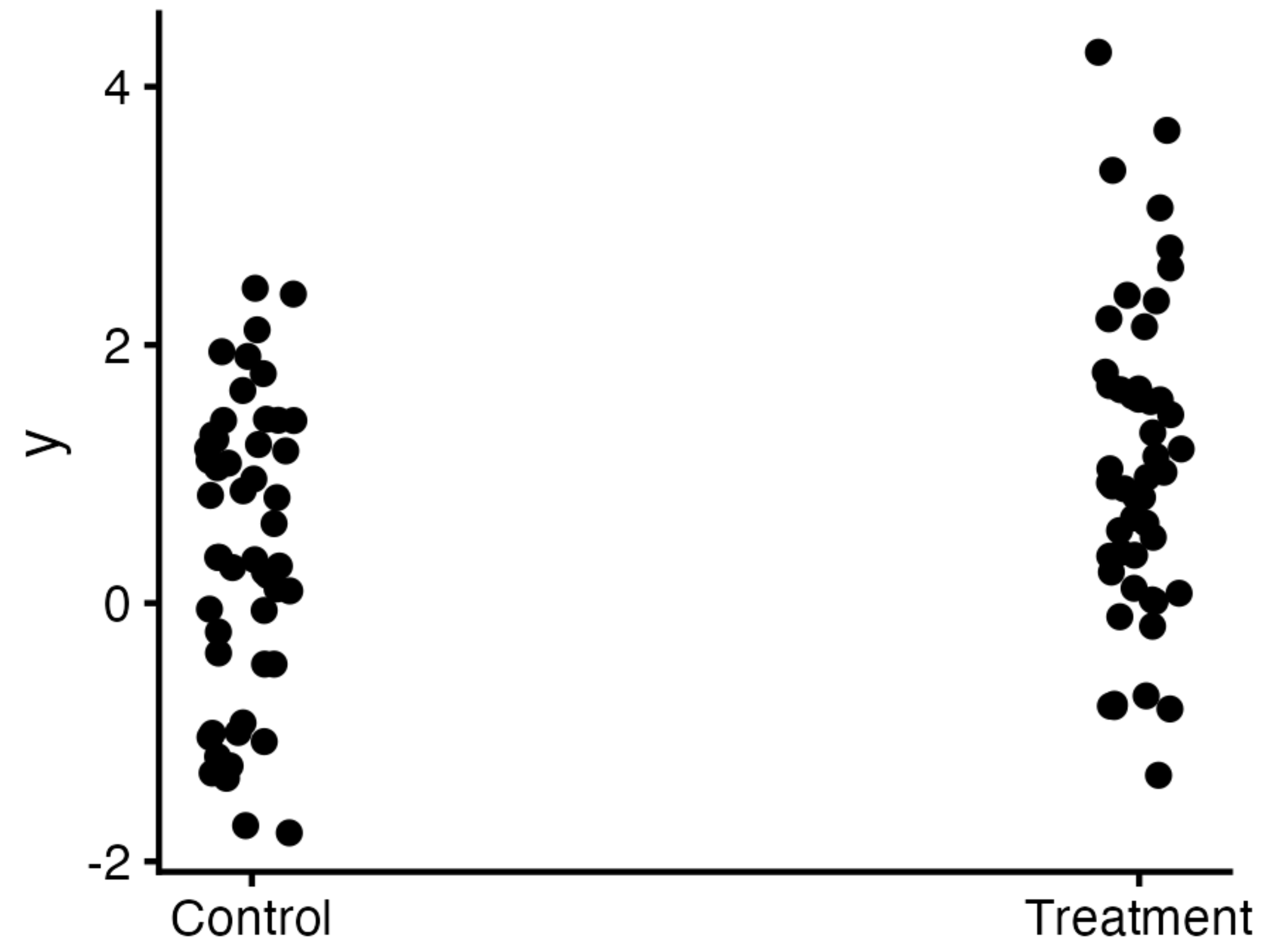
Model	Dependent variable	Independent Variable	Interpretation of β
Level-level	y	x	$\Delta y \approx \beta \Delta x$
Level-log	y	log(x)	$\Delta y \approx (\frac{\beta}{100}) \% \Delta x$
log-level	log(y)	x	$\% \Delta y \approx (100 \beta) \Delta x$
Log-log	log(y)	log(x)	$\% \Delta y \approx \beta \% \Delta x$

Categorical predictors

What about categorical predictors?

Linear regression is flexible

- Our questions are frequently based on categories:
 - Is a treatment effective in improving outcomes?
 - Are two geographical regions different in some aspect?
 - Does the diet of a group of species affect their size?



Binary predictor, same model

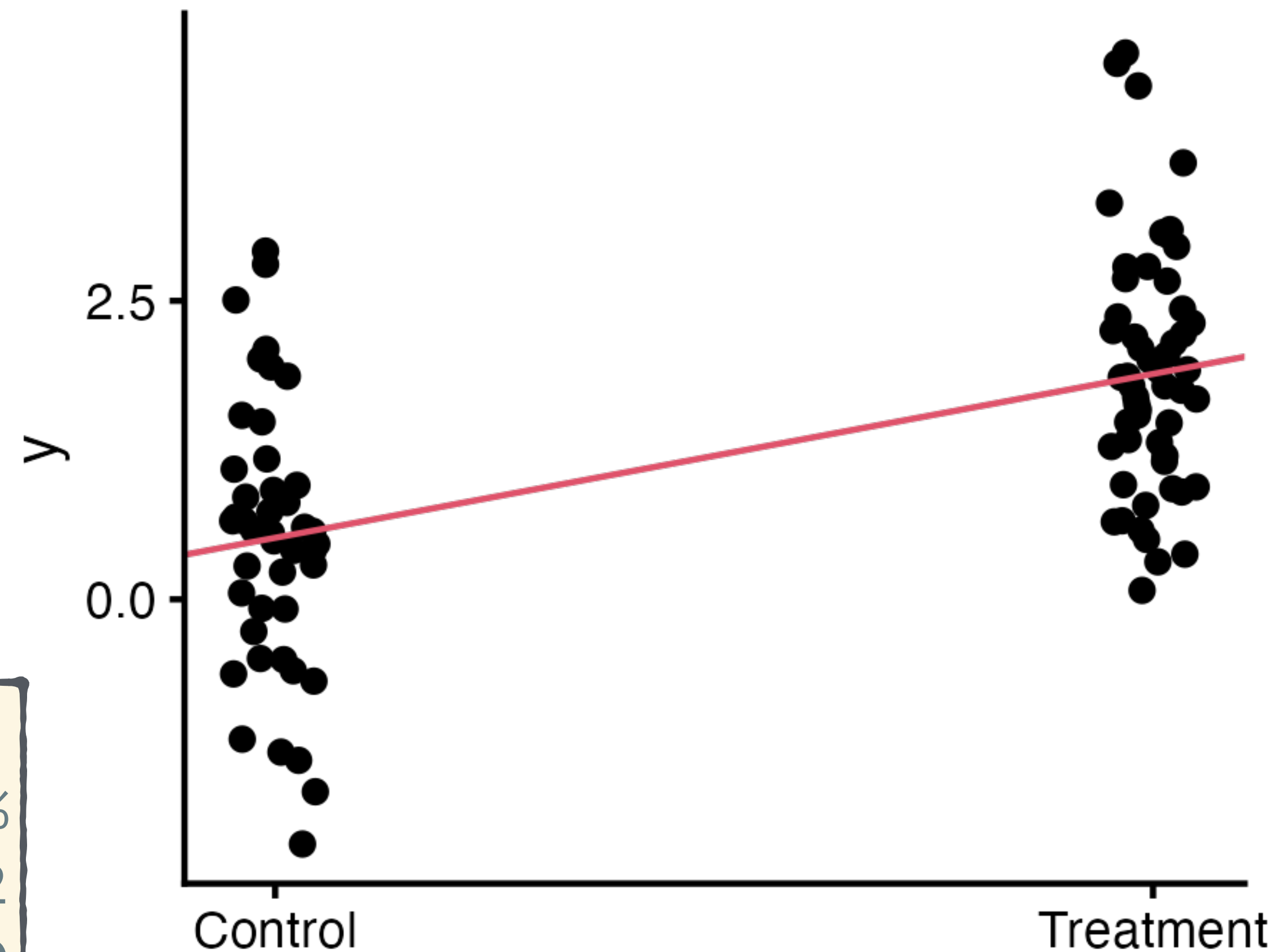
Control-treatment, two categories...

$$y_i \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$

- x_i : 0 for control, 1 for treatment
- α : the intercept is the **mean of the Control** group
- β : the slope is the **difference in the means across the groups**

```
> precis(fit, prob = 0.95)
      mean    sd 2.5% 97.5%
a      0.52 0.16 0.20  0.82
b      1.37 0.22 0.96  1.79
sigma 1.08 0.08 0.94  1.23
```



How about more categories?

- There are a few ways of modeling predictors with many categories:
 - **Contrasts**: Each category is compared to a **baseline**, and the coefficients are comparisons between baseline and levels
 - **One-hot**: coefficients are means of each level of the predictor
 - **Residuals**: an overall mean is measured, and coefficients are differences between each level and the global mean

```
> x = sample(LETTERS[1:3], 9, replace = TRUE)
> y = 1 + ifelse(x == "A", 0,
                ifelse(x == "B", 1, 2)) + rnorm(9)
> df = tibble(y, x)
> df
# A tibble: 9 × 2
      y x
  <dbl> <chr>
1  1.25 B
2  0.870 A
3 -0.00180 A
4  1.18 B
5  2.03 C
6  2.60 B
7  2.55 B
8  3.92 C
9  4.66 B
```

How about more categories?

Contrasts, the default in most `lm()` functions

- There are a few ways of modeling predictors with many categories:
 - **Contrasts:** Each category is compared to a **baseline**, and the coefficients are comparisons between baseline and levels
 - **One-hot:** coefficients are means of each level of the predictor
 - **Residuals:** an overall mean is measured, and coefficients are differences between each level and the global mean

```
> fit_contrasts = stan_glm(y ~ x, data = df)
> summary(fit_contrasts)[1:3, 1:3]
```

	mean	mcse	sd
(Intercept)	0.72	0	0.13
xB	1.32	0	0.20
xC	2.20	0	0.186

Q: How do we get estimates for the mean in each class?

How about more categories?

One-hot

- There are a few ways of modeling predictors with many categories:
 - **Contrasts**: Each category is compared to a **baseline**, and the coefficients are comparisons between baseline and levels
 - **One-hot**: coefficients are means of each level of the predictor
 - **Residuals**: an overall mean is measured, and coefficients are differences between each level and the global mean

```
> x
[1] "B" "A" "A" "B" "C" "B" "B" "C" "B"
> onehot
  xA  xB  xC
1  0  1  0
2  1  0  0
3  1  0  0
4  0  1  0
5  0  0  1
6  0  1  0
7  0  1  0
8  0  0  1
9  0  1  0
```

How about more categories?

One-hot

- There are a few ways of modeling predictors with many categories:
 - **Contrasts**: Each category is compared to a **baseline**, and the coefficients are comparisons between baseline and levels
 - **One-hot**: coefficients are means of each level of the predictor
 - **Residuals**: an overall mean is measured, and coefficients are differences between each level and the global mean

```
> onehot = model.matrix(~0+x, data = df)
> fit_onehot = stan_glm(y ~ 0 + onehot, data =
df, cores = 4)
> summary(fit_onehot)[1:3, 1:3]
```

	mean	mcse	sd
onehotxA	0.72	0	0.13
onehotxB	2.04	0	0.14
onehotxC	2.93	0	0.12

How about more categories?

One-hot

- There are a few ways of modeling predictors with many categories:
 - **Contrasts:** Each category is compared to a **baseline**, and the coefficients are comparisons between baseline and levels
 - **One-hot:** coefficients are means of each level of the predictor
 - **Residuals:** an overall mean is measured, and coefficients are differences between each level and the global mean

```
> onehot = model.matrix(~0+x, data = df)
> fit_onehot = stan_glm(y ~ 0 + onehot, data = df, cores = 4)
> summary(fit_onehot)[1:3, 1:3]
```

	mean	mcse	sd
onehotxA	0.72	0	0.13
onehotxB	2.04	0	0.14
onehotxC	2.93	0	0.12

```
> fit_contrasts = stan_glm(y ~ x, data = df)
> summary(fit_contrasts)[1:3, 1:3]
```

	mean	mcse	sd
(Intercept)	0.72	0	0.13
xB	1.32	0	0.20
xC	2.20	0	0.186

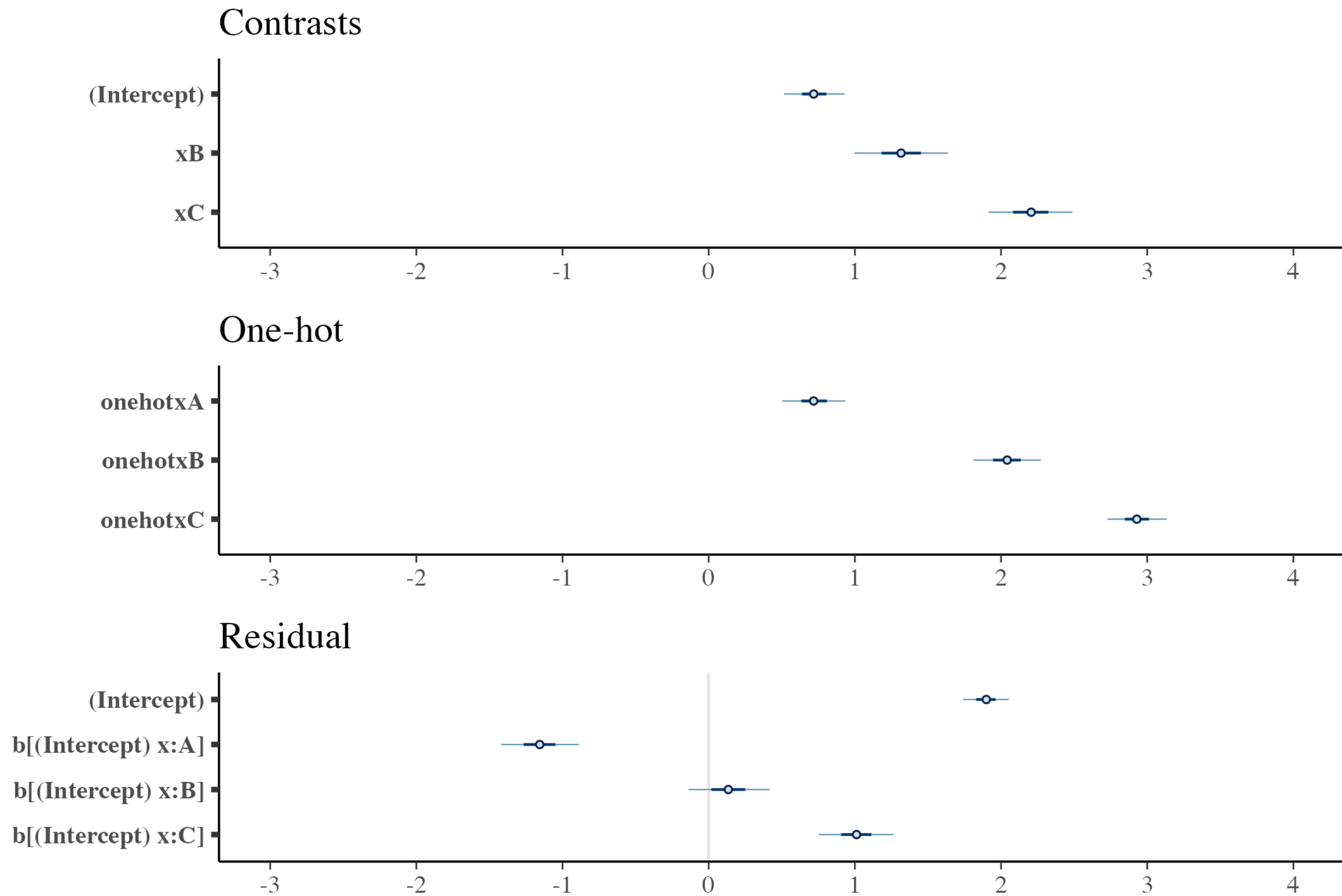
How about more categories?

- There are a few ways of modeling predictors with many categories:
 - **Contrasts:** Each category is compared to a **baseline**, and the coefficients are comparisons between baseline and levels
 - **One-hot:** coefficients are means of each level of the predictor
 - **Residuals:** an overall mean is measured, and coefficients are differences between each level and the global mean

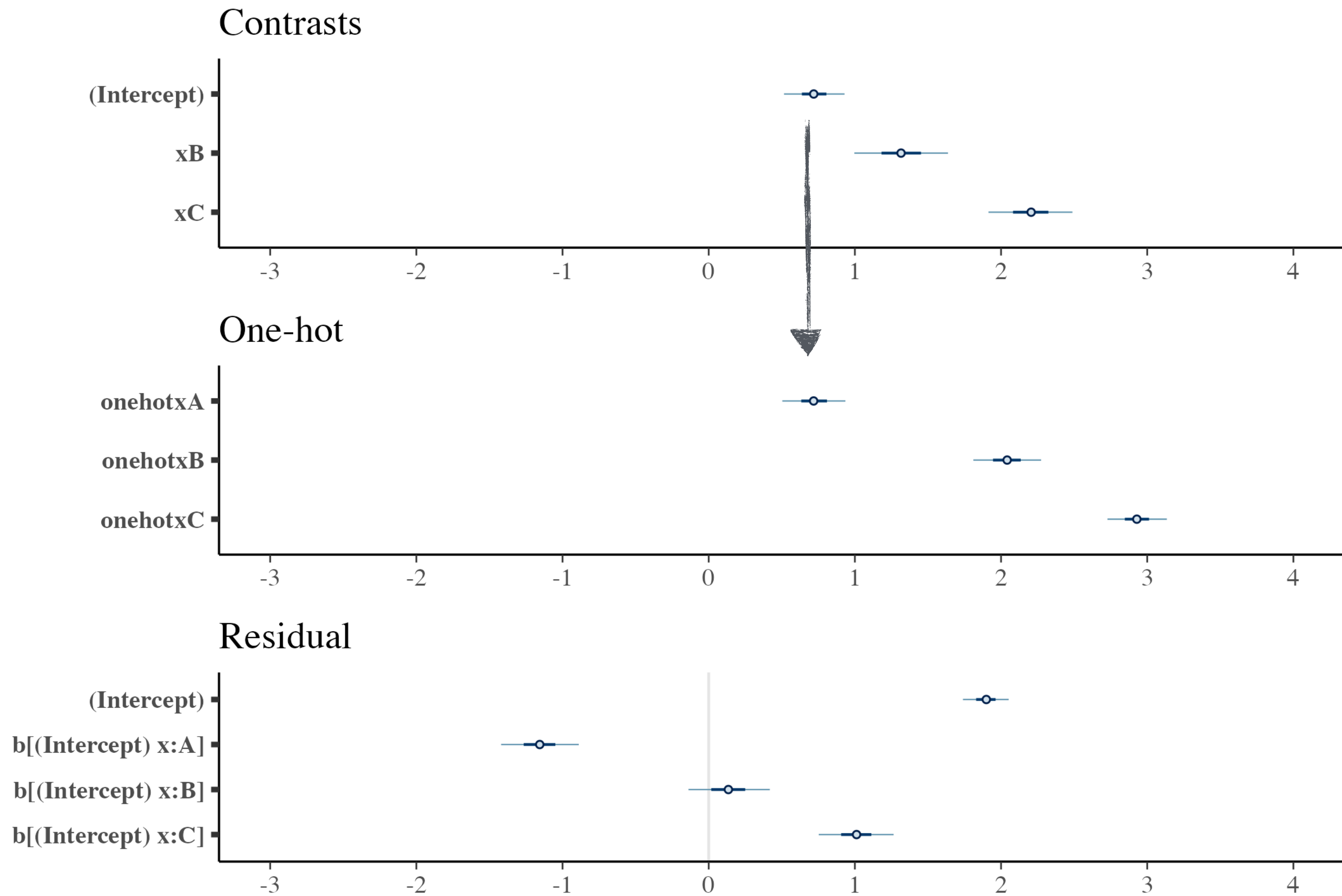
```
> fit_residual = stan_glmer(y ~ 1 + (1|x),
data = df, prior_intercept =
normal(mean(df$y), 0.1))
> summary(fit_residual)[1:4, 1:3]
```

	mean	mcse	sd
(Intercept)	1.90	0	0.10
b[(Intercept) x:A]	-1.16	0	0.16
b[(Intercept) x:B]	0.14	0	0.17
b[(Intercept) x:C]	1.01	0	0.15

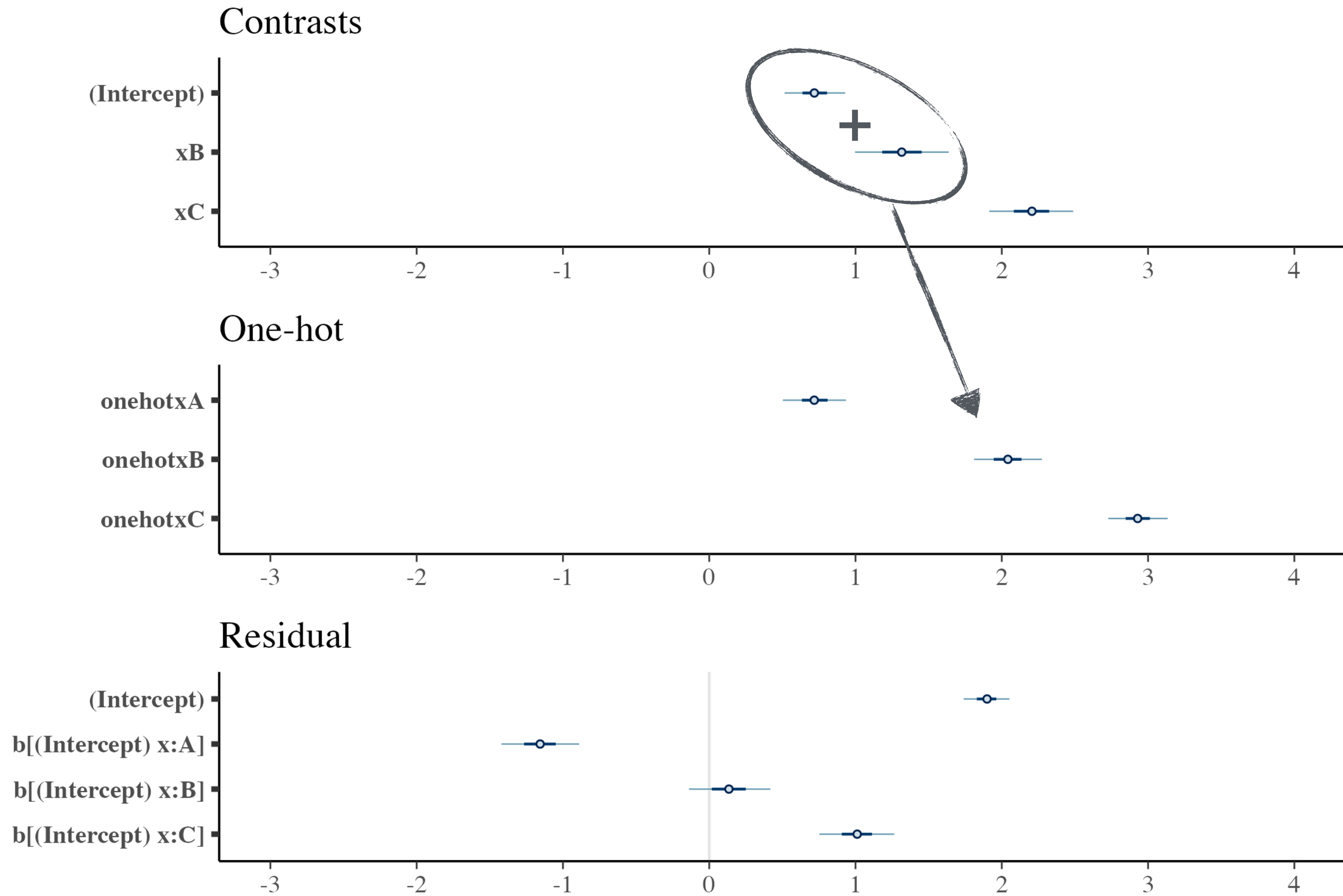
Comparing estimates



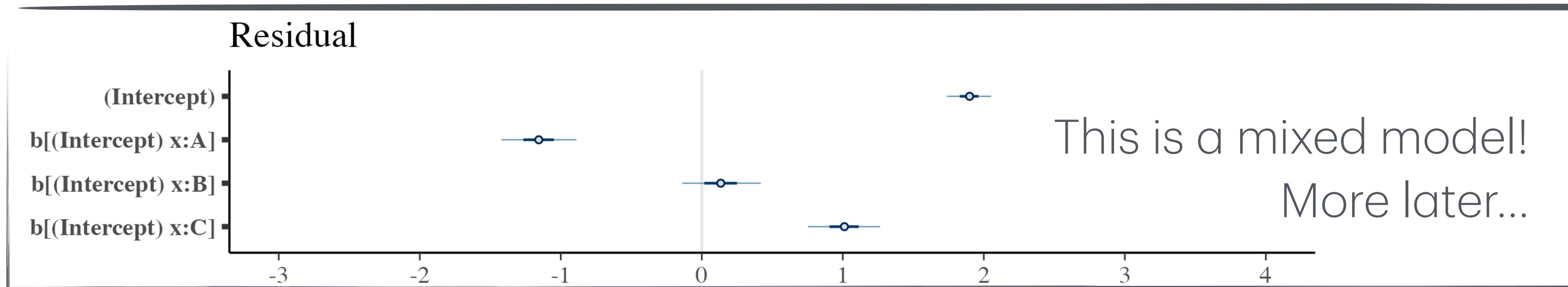
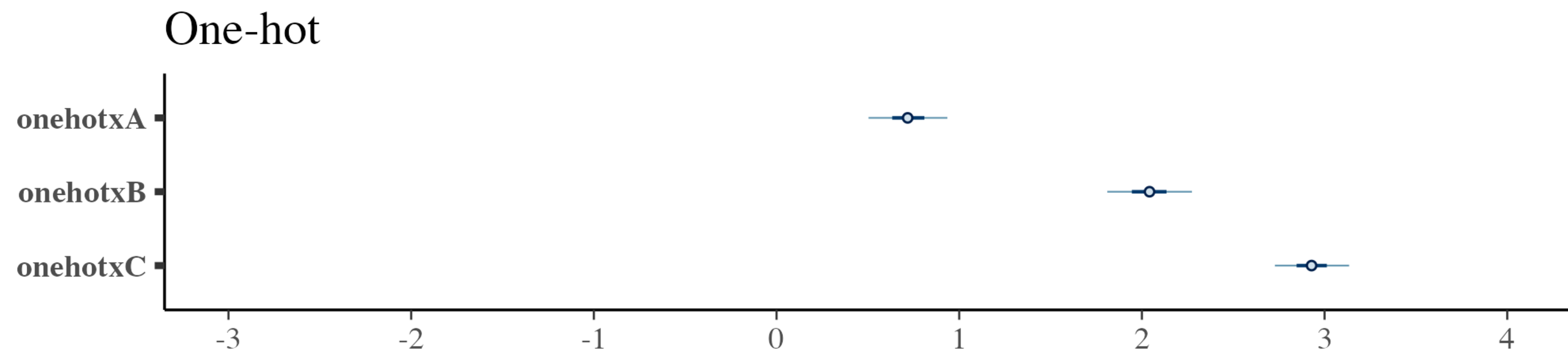
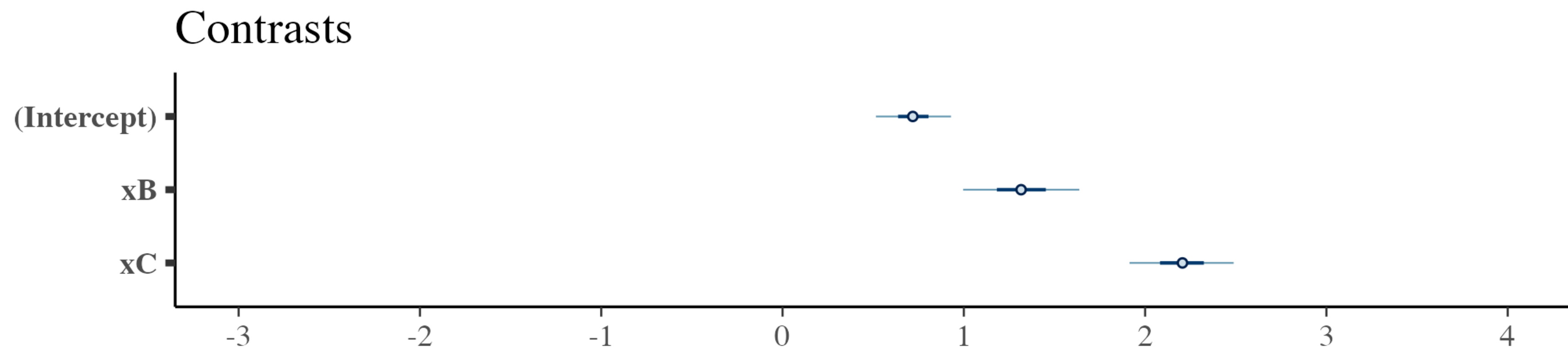
Comparing estimates



Comparing estimates



Comparing estimates



Multiple predictors

Linear regression is flexible!

We can modify our functions however we like

- The linear model we are using consists of making the parameters of probability distributions change according to some function
- The simplest function involves a single predictor and slope
- If we have more predictors, we can simply add them to the regression equation

$$y_i \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$

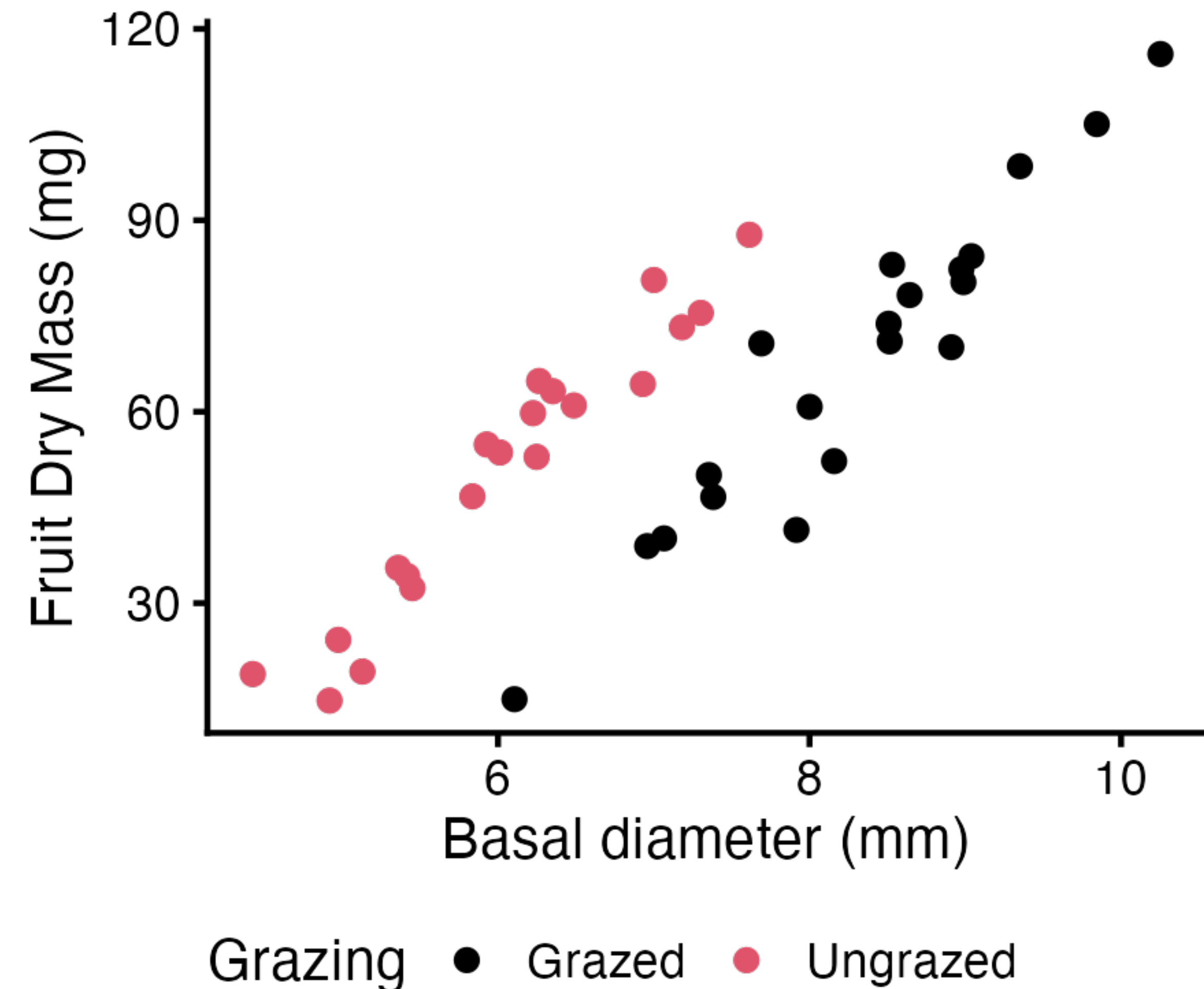


$$y_i \sim N(\mu_i, \sigma)$$

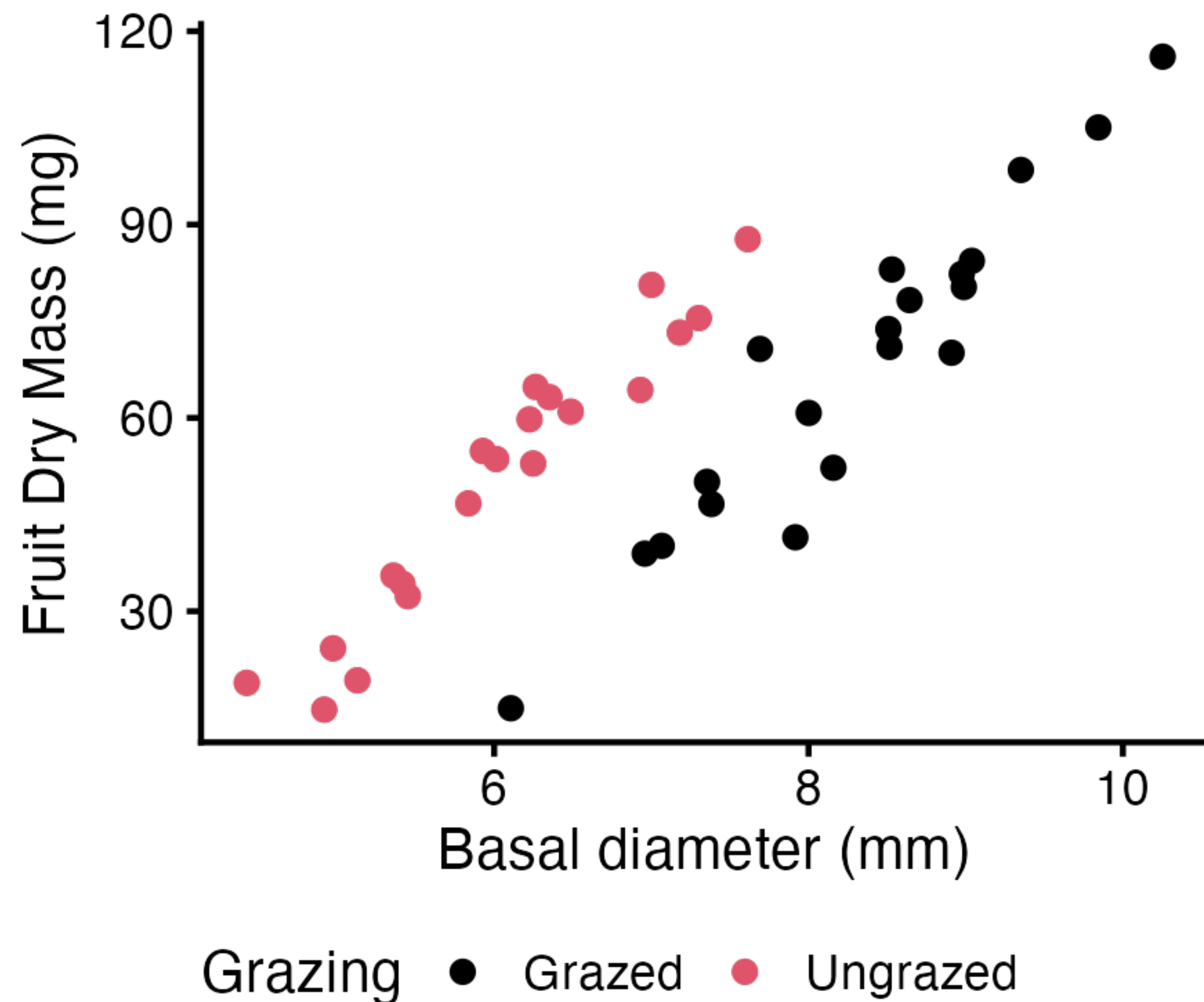
$$\mu_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2}$$

Example with more predictors

- **Question:** what's the impact of herbivory on plant fitness?
- **Field experiment:** 40 plants of *Ipomopsis aggregata* assigned at random to two treatments: unprotected from grazing by rabbits and protect from grazing by fenced cages.
- Response variable: fruit yield of each plant (mg dry mass)
- Predictor variables:
 - Treatment: fenced or non-fenced
 - Basal diameter of each plant (mm), measured before the treatment



Scale variables



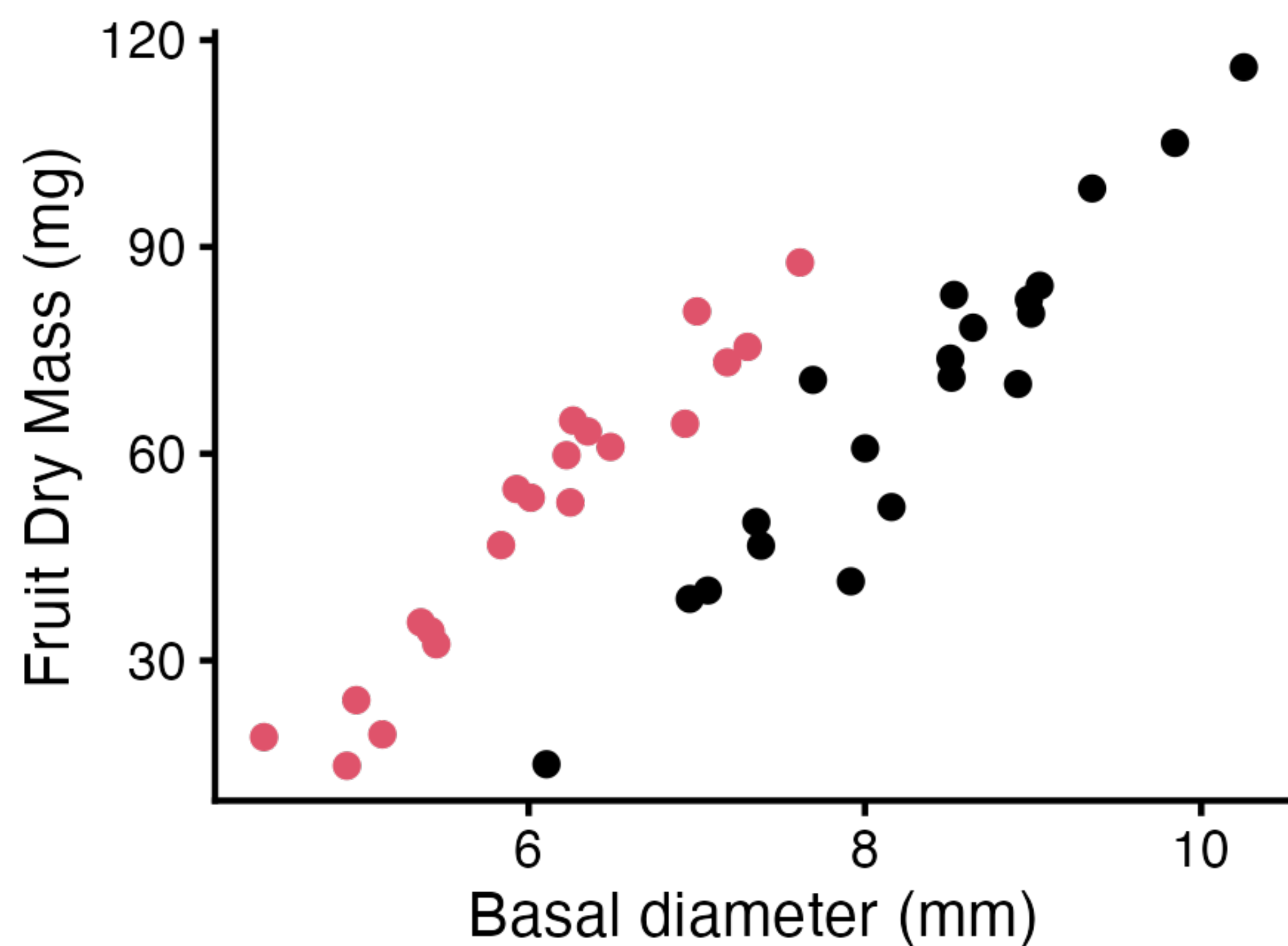
- It's good practice to scale variables by their standard deviation and subtract the mean:

$$\tilde{y}_i = \frac{y_i - \bar{y}}{sd(y)}$$

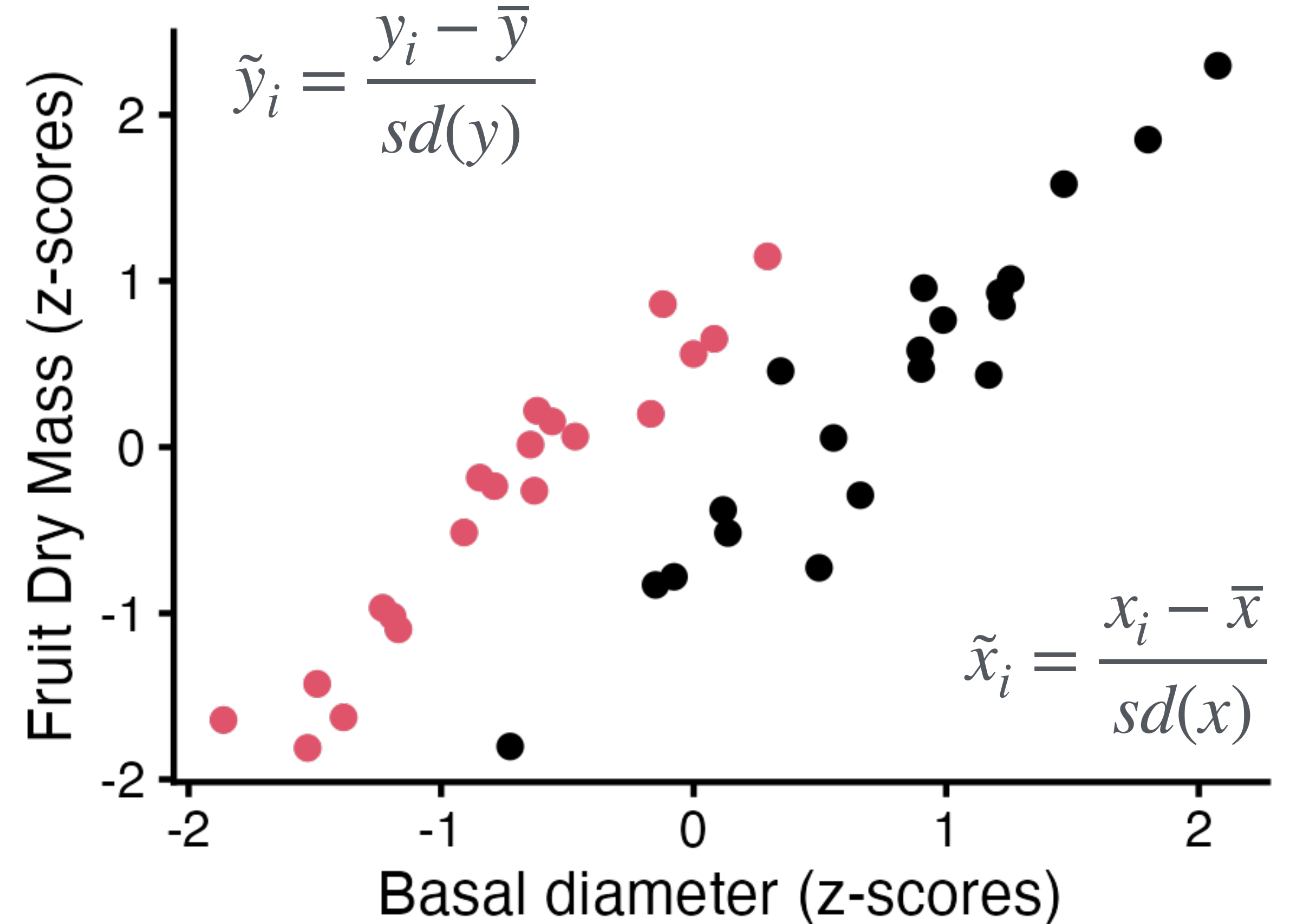
- The **z-score** of a continuous variable is a measure of how many standard deviations a data point is from the mean of the dataset
- Using z-scores makes coefficients easier to interpret and comparable across variables with different scales
- The transformation is linear, and we can always recover parameter values on their original scale by multiplying by the standard deviation

Scale variables

Using standard deviation units makes everything simpler



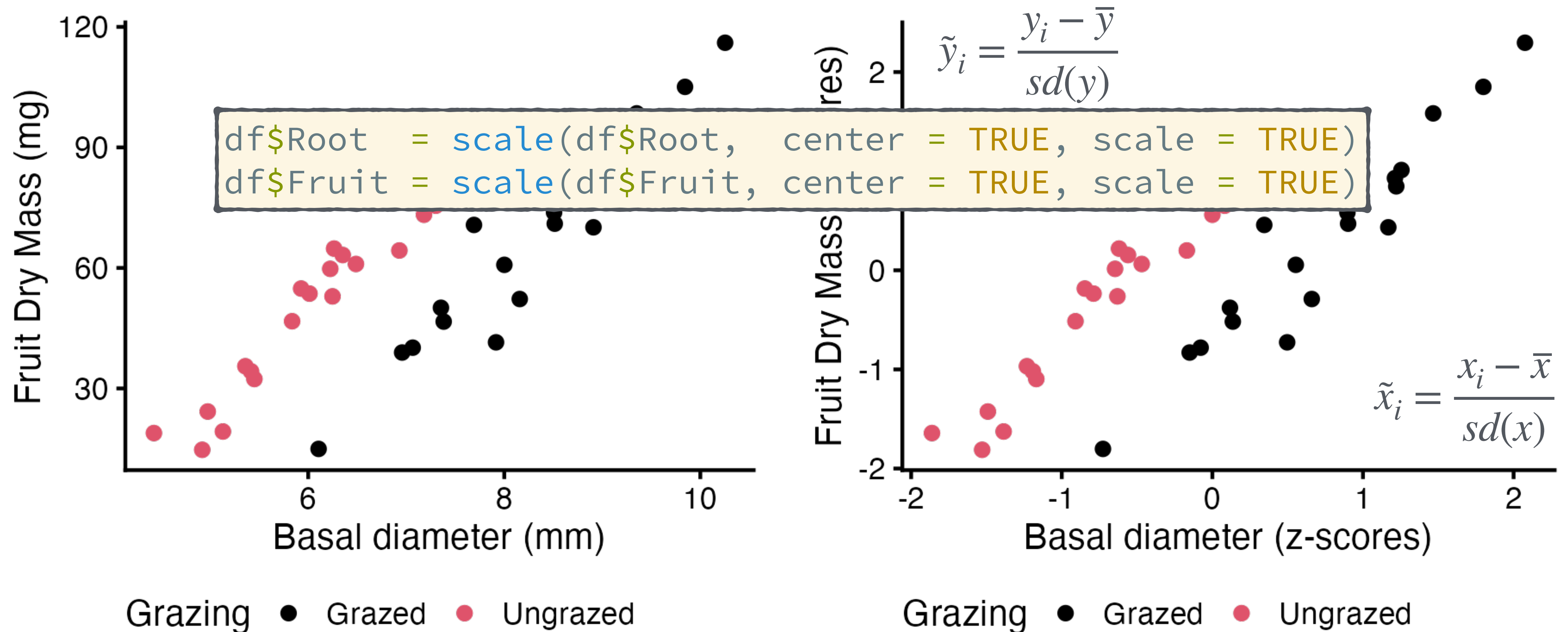
Grazing ● Grazed ● Ungrazed ●



Grazing ● Grazed ● Ungrazed ●

Scale variables

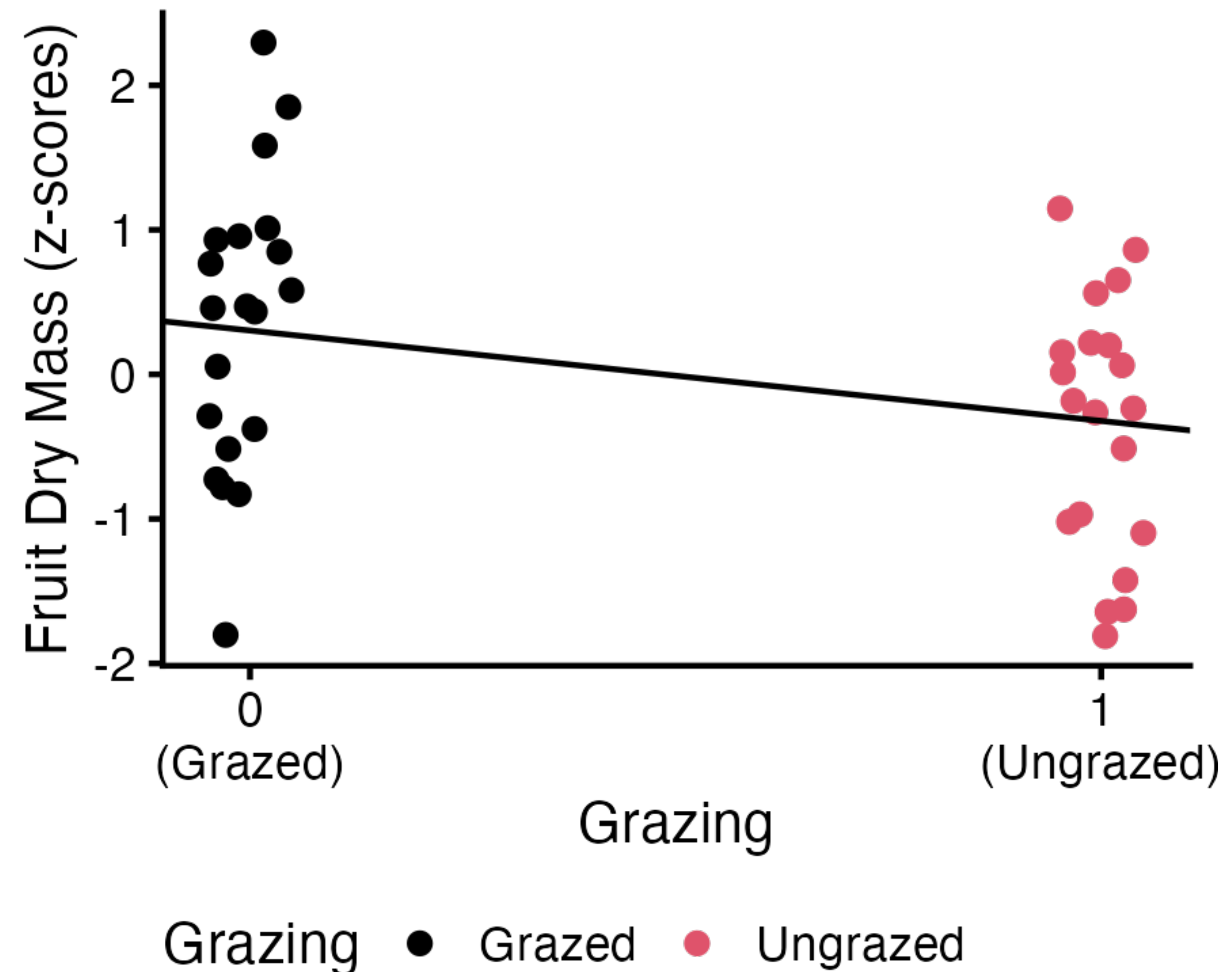
Using standard deviation units makes everything simpler



Model with only treatment

```
df$Grazing0 = ifelse(df$Grazing ==  
"Ungrazed", 1, 0) # numeric coding  
  
m1 = quap(alist(  
  Fruit ~ normal(mu, sigma),  
  mu <- a + b*Grazing0),  
  data = df)
```

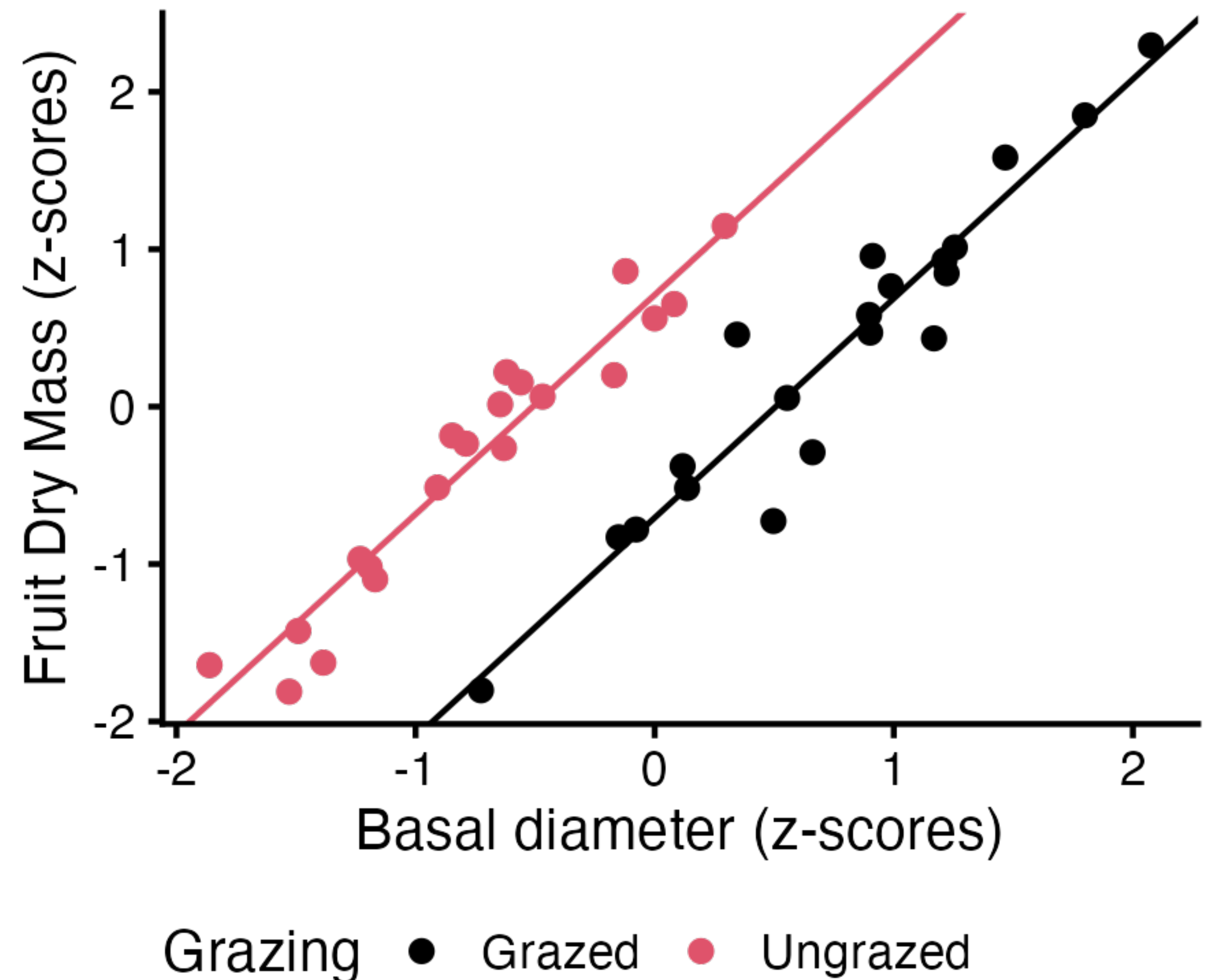
```
> precis(m1, prob = 0.95)  
      mean    sd  2.5% 97.5%  
a      0.32 0.21 -0.09  0.72  
b     -0.66 0.30 -1.23 -0.06  
sigma  0.97 0.12  0.77  1.22
```



Model with treatment and size

```
df$Grazing0 = ifelse(df$Grazing ==  
"Ungrazed", 1, 0) # numeric coding  
  
m1 = quap(alist(  
  Fruit ~ normal(mu, sigma),  
  mu <- a + b*Grazing0),  
  data = df)
```

```
> precis(m2, prob = 0.95)  
      mean    sd 2.5% 97.5%  
a    -0.71 0.08 -0.86 -0.54  
b     1.42 0.14  1.13  1.67  
c     1.39 0.07  1.26  1.53  
sigma 0.28 0.03  0.23  0.36
```

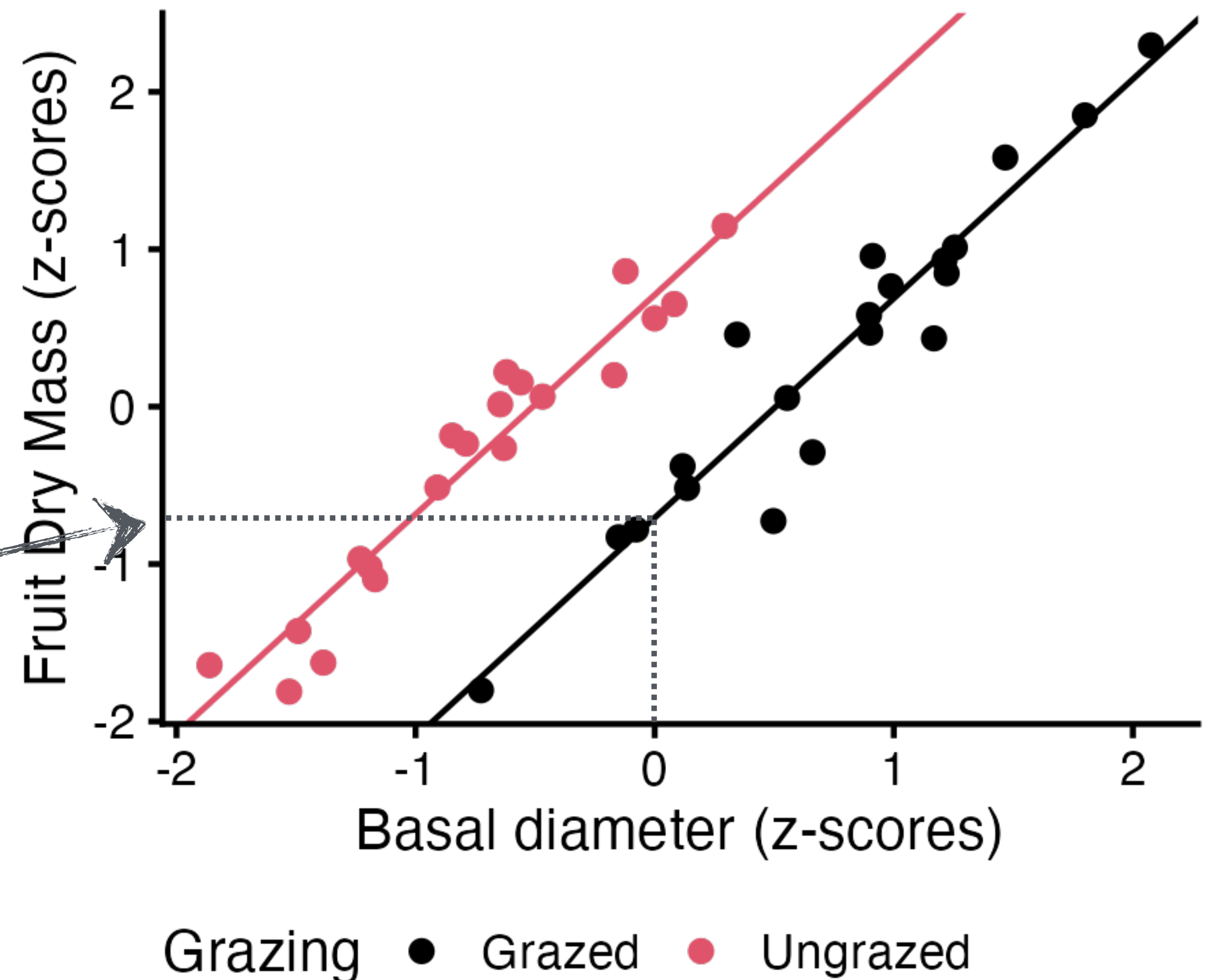


Model with treatment and size

```
m2 = ulam(alist(  
  Fruit ~ normal(mu, sigma),  
  mu <- a + b*Grazing0 + c*Root,  
  a ~ normal(0, 1),  
  b ~ normal(0, 1),  
  c ~ normal(0, 1),  
  sigma ~ exponential(1)),  
  data = df, chains = 4, cores = 4)
```

```
> precis(m2, prob = 0.95)
```

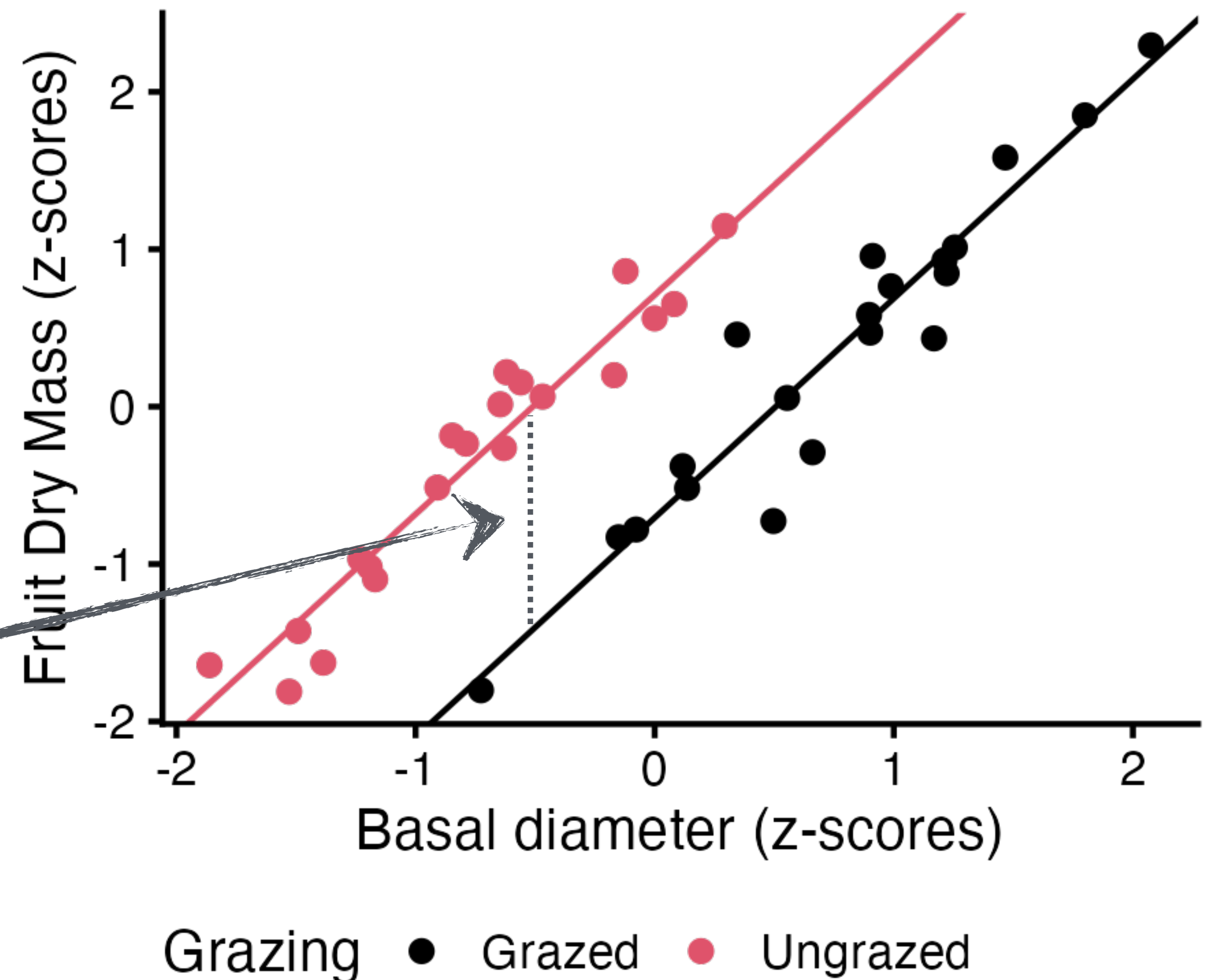
	mean	sd	2.5%	97.5%
a	-0.71	0.08	-0.86	-0.54
b	1.42	0.14	1.13	1.67
c	1.39	0.07	1.26	1.53
sigma	0.28	0.03	0.23	0.36



Model with treatment and size

```
m2 = ulam(alist(  
  Fruit ~ normal(mu, sigma),  
  mu <- a + b*Grazing0 + c*Root,  
  a ~ normal(0, 1),  
  b ~ normal(0, 1),  
  c ~ normal(0, 1),  
  sigma ~ exponential(1)),  
  data = df, chains = 4, cores = 4)
```

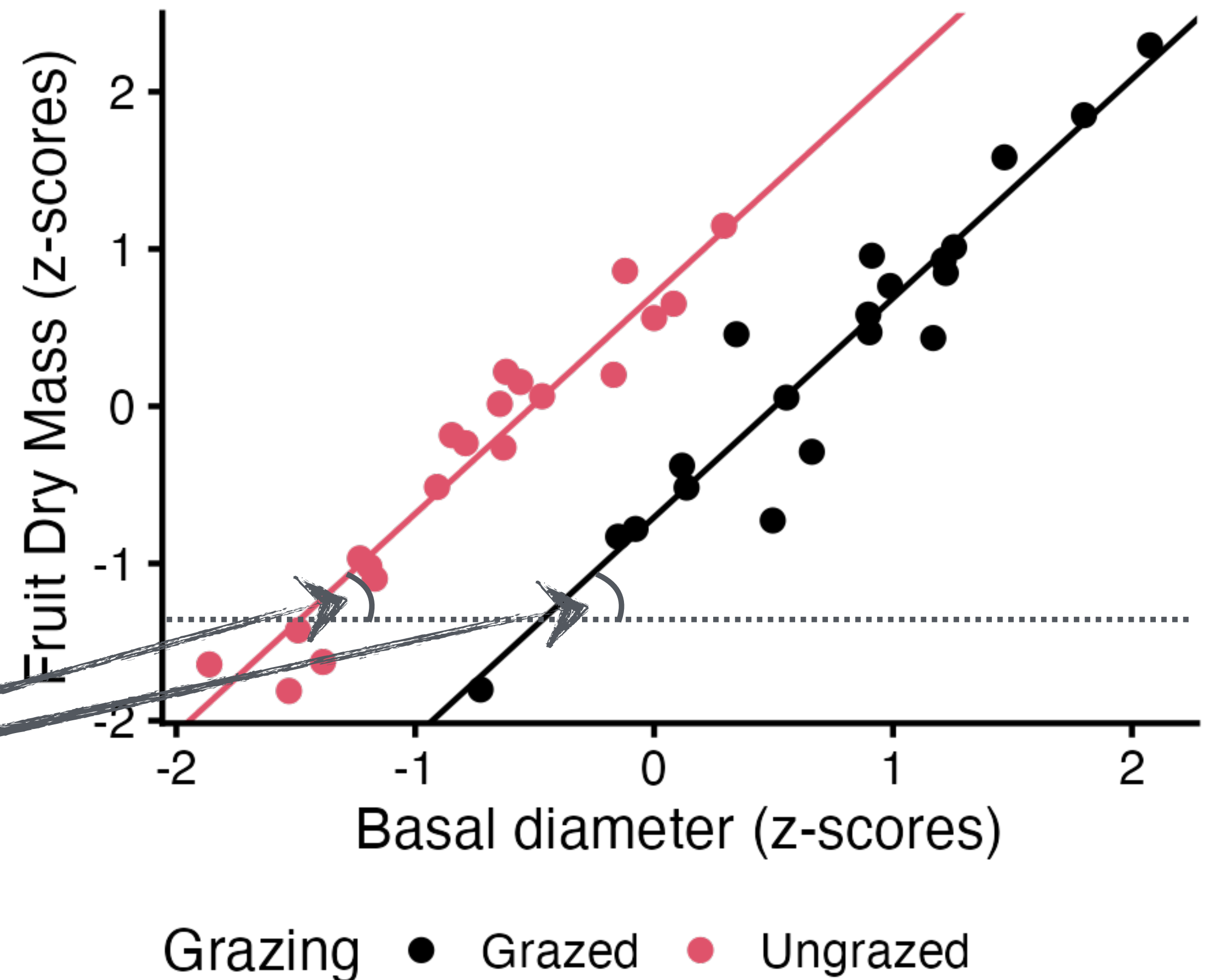
```
> precis(m2, prob = 0.95)  
      mean      sd  2.5% 97.5%  
a    -0.71  0.08  -0.86 -0.54  
b     1.42  0.14   1.13  1.67  
c     1.39  0.07   1.26  1.53  
sigma 0.28  0.03   0.23  0.36
```



Model with treatment and size

```
m2 = ulam(alist(  
  Fruit ~ normal(mu, sigma),  
  mu <- a + b*Grazing0 + c*Root,  
  a ~ normal(0, 1),  
  b ~ normal(0, 1),  
  c ~ normal(0, 1),  
  sigma ~ exponential(1)),  
  data = df, chains = 4, cores = 4)
```

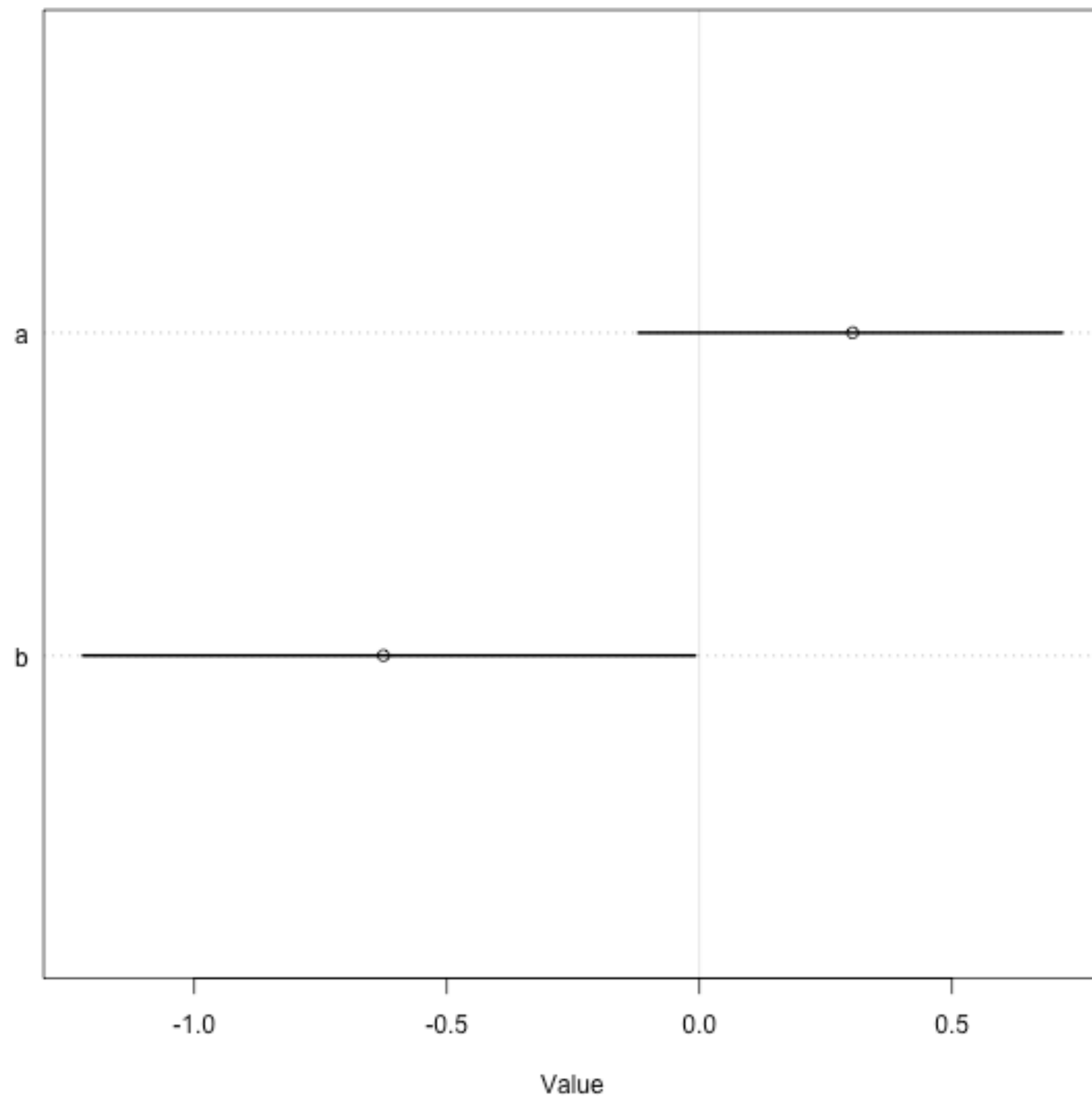
```
> precis(m2, prob = 0.95)  
      mean      sd 2.5% 97.5%  
a    -0.71  0.08 -0.86 -0.54  
b     1.42  0.14  1.13  1.67  
c     1.39  0.07  1.26  1.53  
sigma 0.28  0.03  0.23  0.36
```



Comparing model estimates

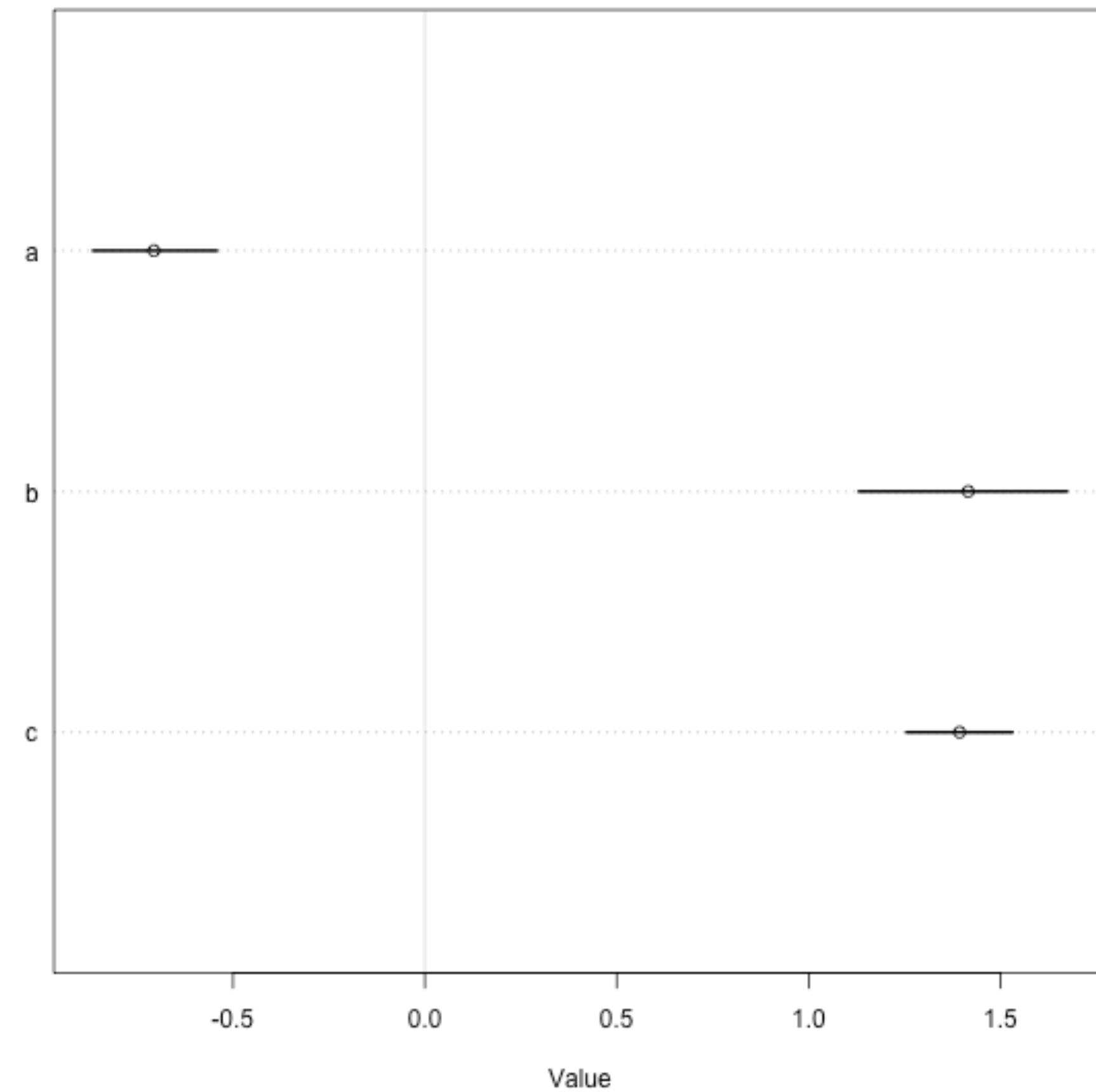
$$y_i \sim N(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta x_i$$



$$y_i \sim N(\mu_i, \sigma)$$

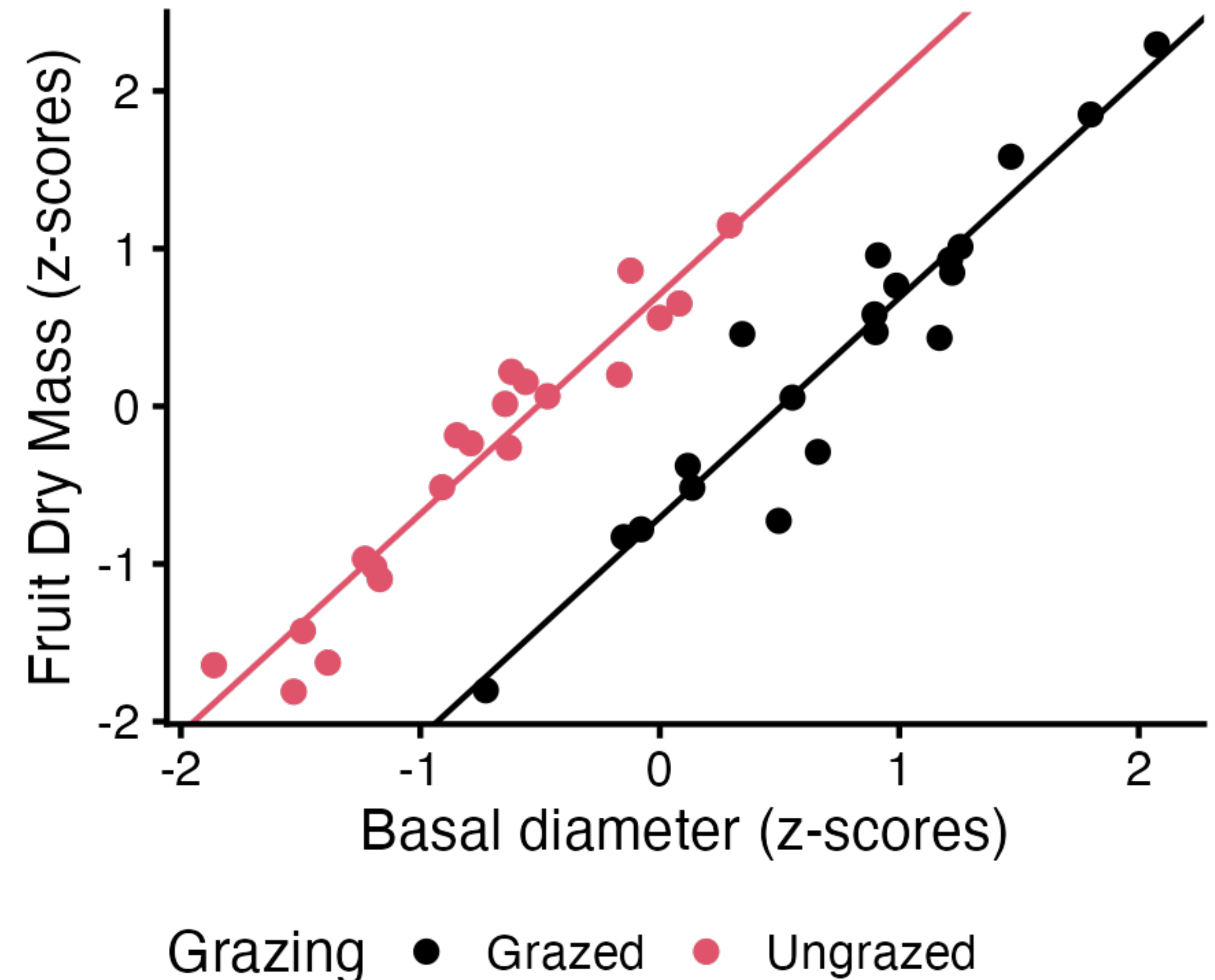
$$\mu_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2}$$



Why do the coefficients change?

Multiple regression as control

- Coefficients are comparisons, and adding predictors to a regression has the same effect as **stratifying** the data
- The objective of adding more predictors to a linear model is to compare **like-to-like**:
 1. What is the difference between treatments for plants of the same initial size?
 2. What is the effect of initial size given a particular treatment?



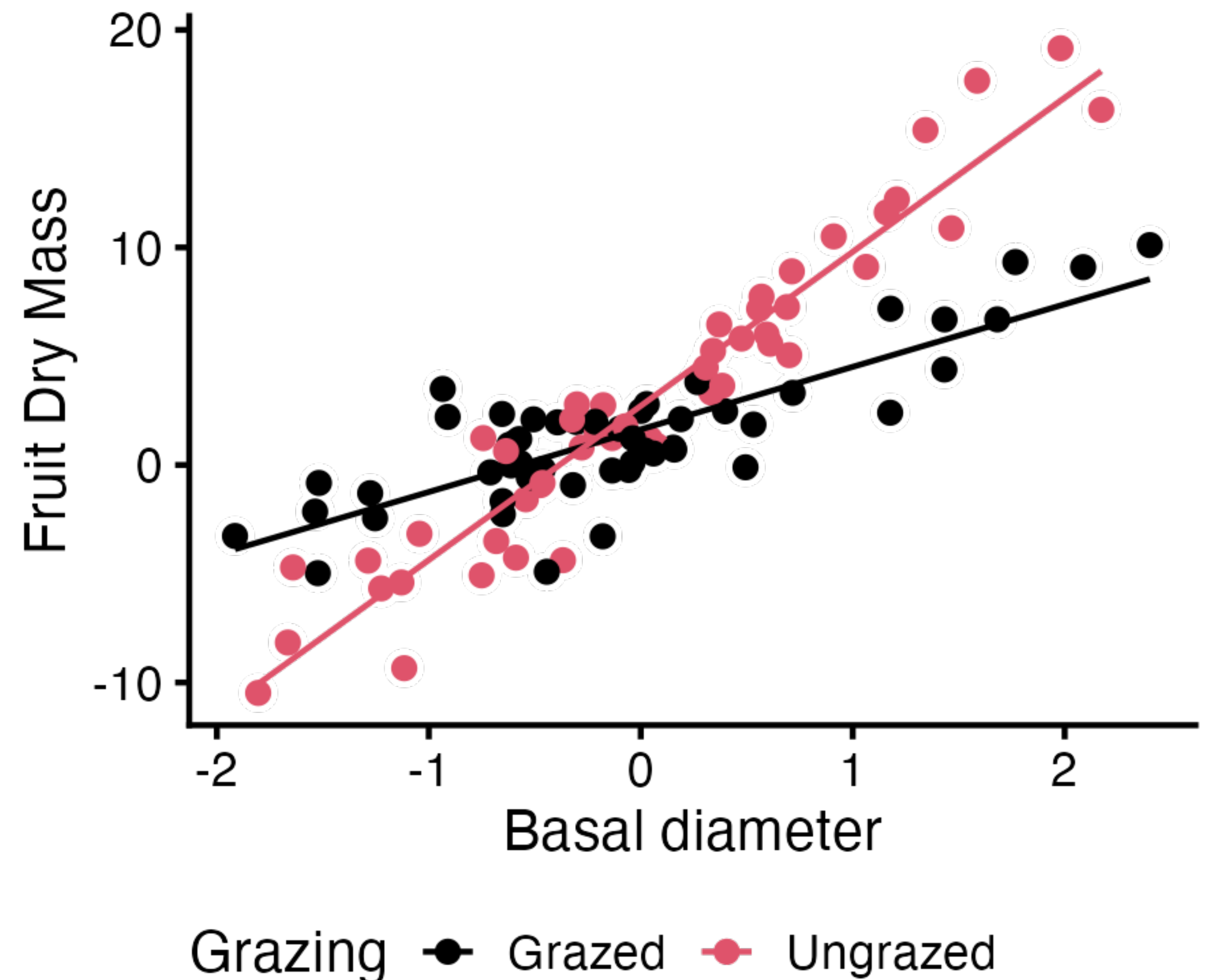
Interactions

Interactions

Allowing coefficients to change according to other variables

- What if the relation between a predictor and an outcome changes depending on another predictor?
- We can account for this by adding product terms in the model
- With two predictors, x and z :

$$\mu = \alpha + \beta_1 x + \beta_2 z + \beta_3 xz$$



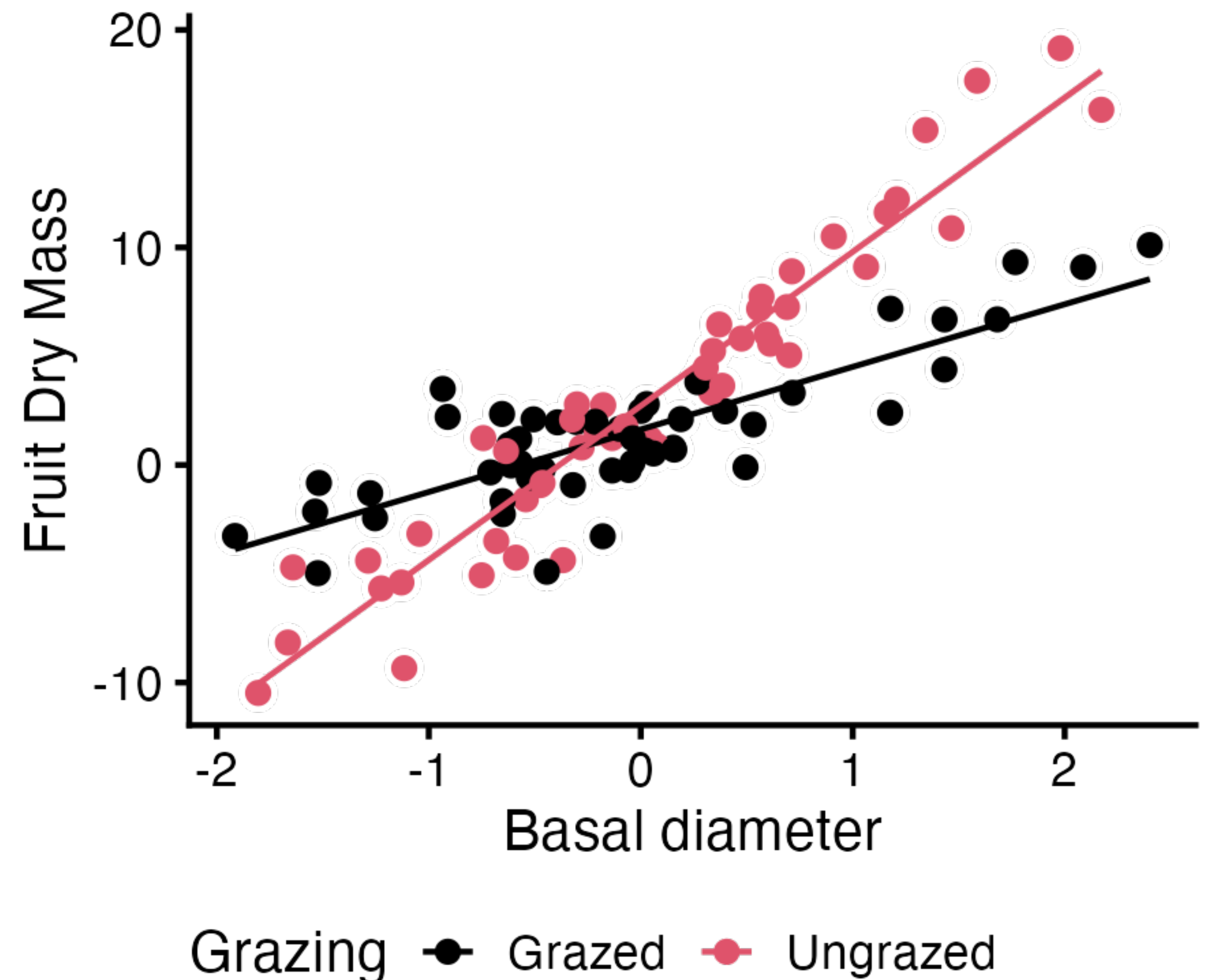
Interactions

Allowing coefficients to change according to other variables

- What if the relation between a predictor and an outcome changes depending on another predictor?
- We can account for this by adding product terms in the model
- With two predictors, x and z :

$$\mu = \alpha + \beta_1 x + \beta_2 z + \beta_3 xz$$

Interaction term!



Example with interactions

- **Question:** what are the best soil moisture and light availability to grow tulips?
- Greenhouse experiment: beds of tulips kept in nine combinations of soil moisture and shading. Three replicates for each combination (total of 27 beds).
- **Response variable:** mean plant height in each bed
- **Predictor variables:**
 - Water: 3 levels (low, med, high)
 - Shade: 3 levels (low, med, high)



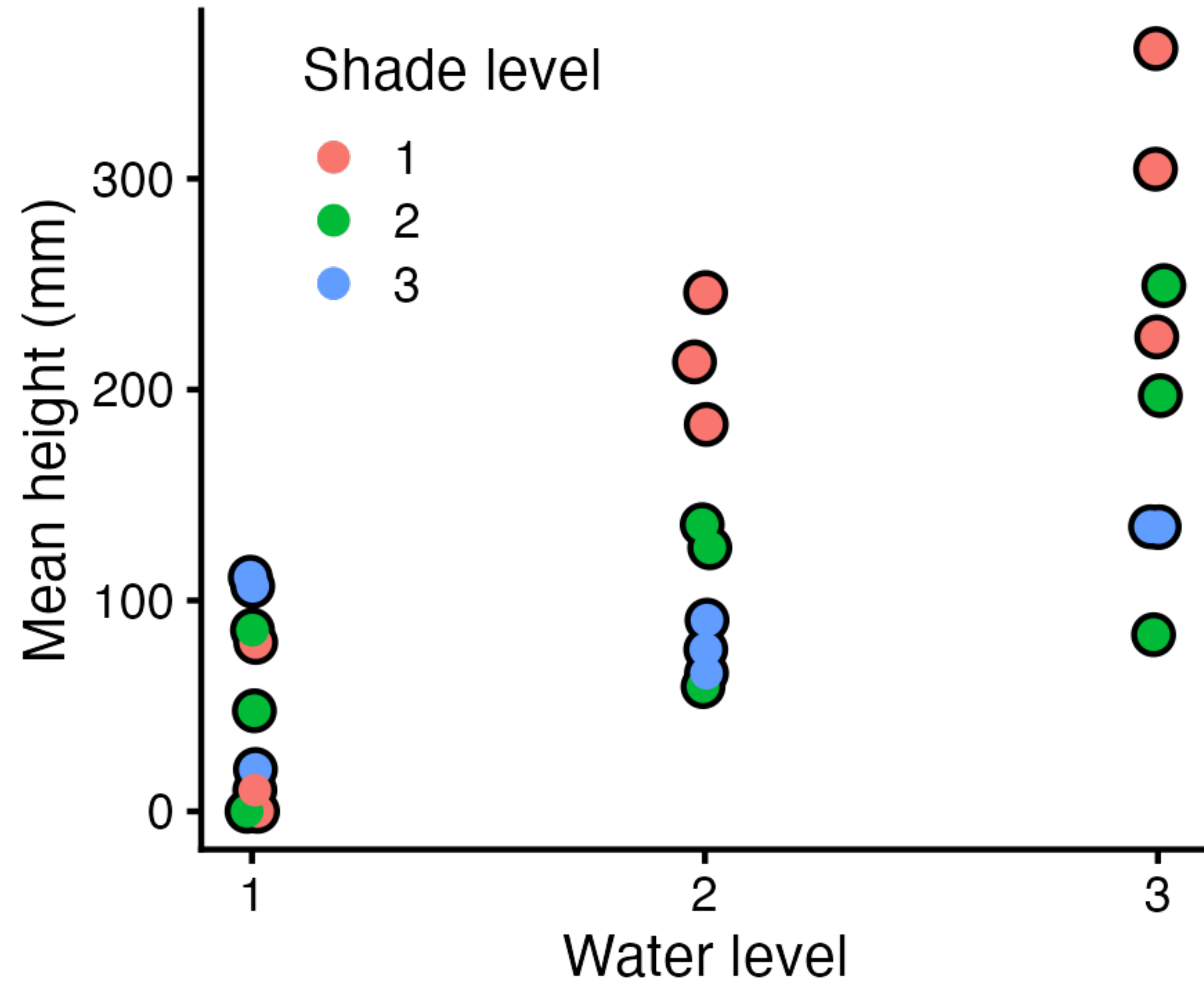
Tulips data

Shade level:

1. Low
2. Medium
3. High

Water level:

1. Low
2. Medium
3. High



Quick transformations

Always think about a scale that makes the model easier to interpret!

Shade level:

-1 : Low

0: Medium

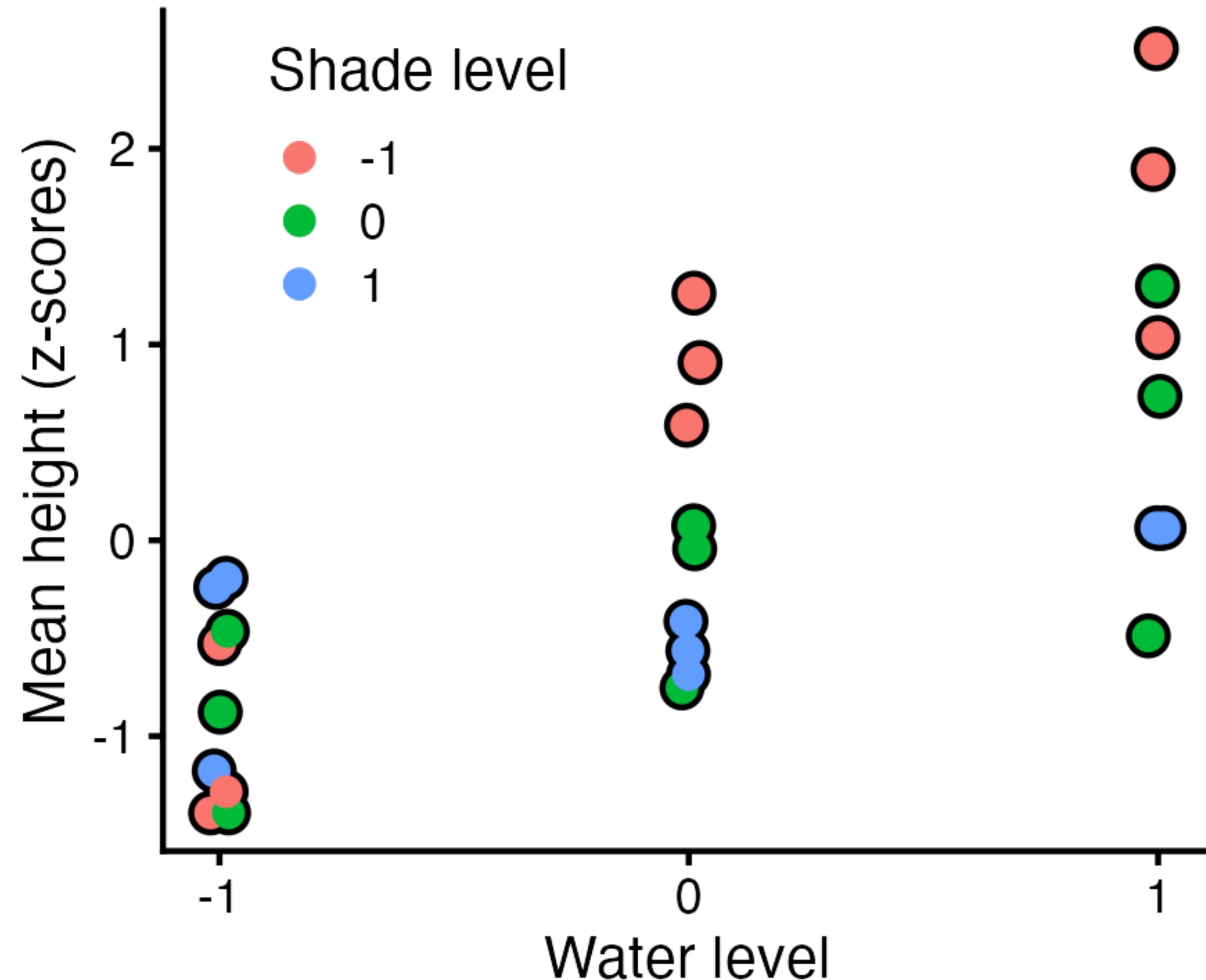
1: High

Water level:

-1 : Low

0: Medium

1: High



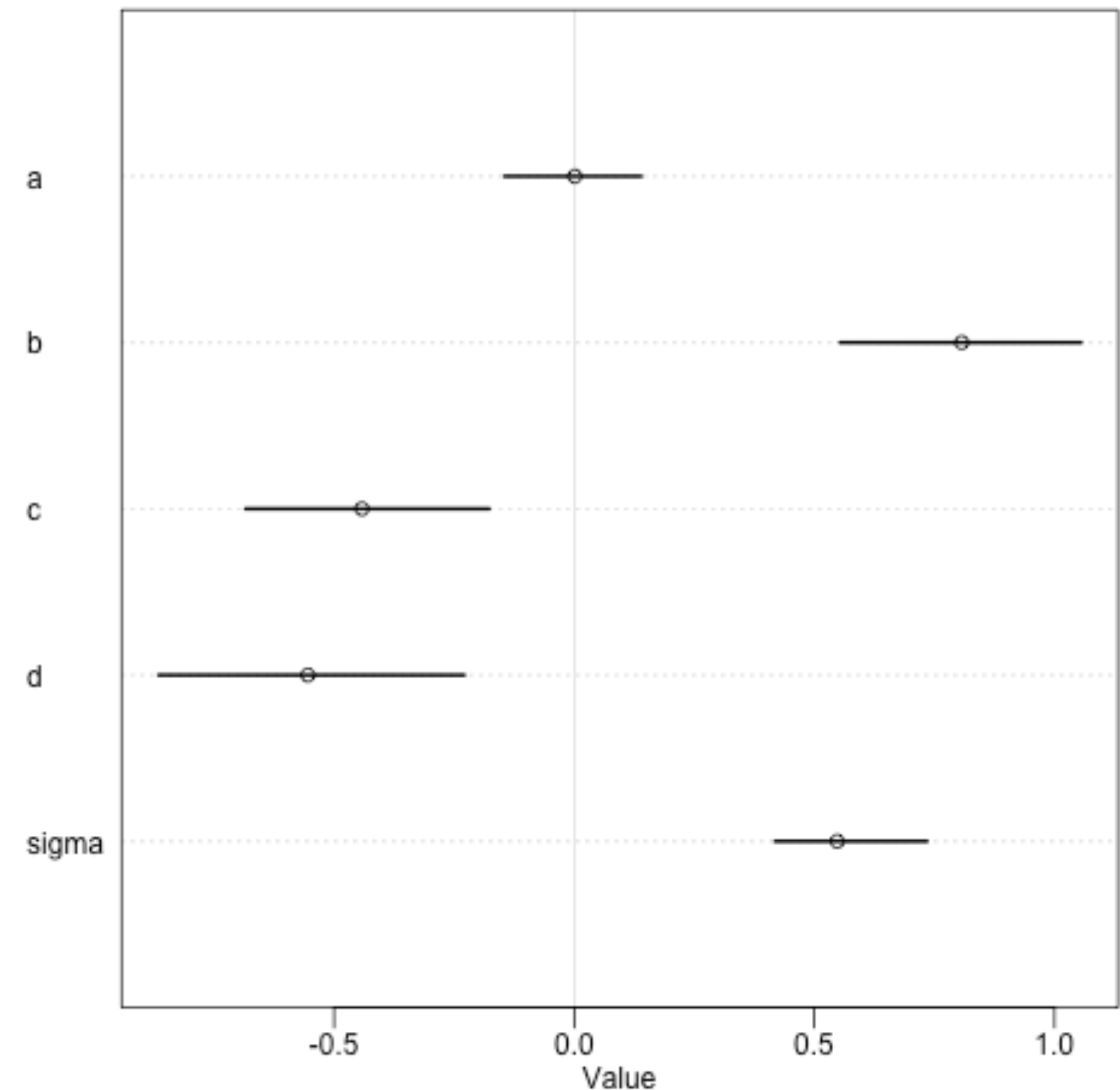
Model with interaction

$Height_i \sim Normal(\mu_i, \sigma)$

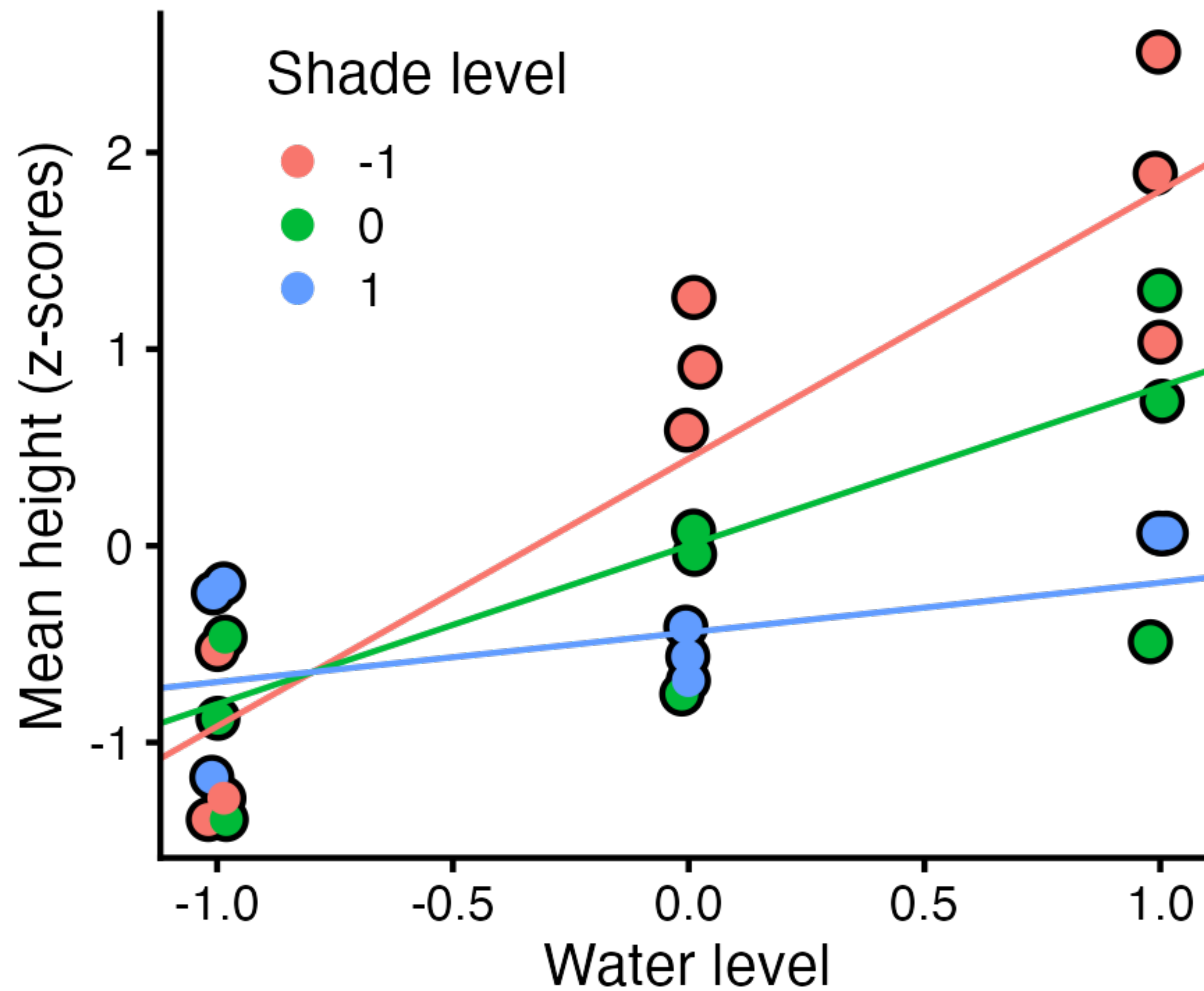
$$\mu_i = a + b \text{ water}_i + c \text{ shade} + d \text{ water}_i \times \text{shade}_i$$

```
data("tulips")
df = tulips
df$water  = scale(df$water, scale = FALSE)
df$shade  = scale(df$shade, scale = FALSE)
df$blooms = scale(df$blooms)

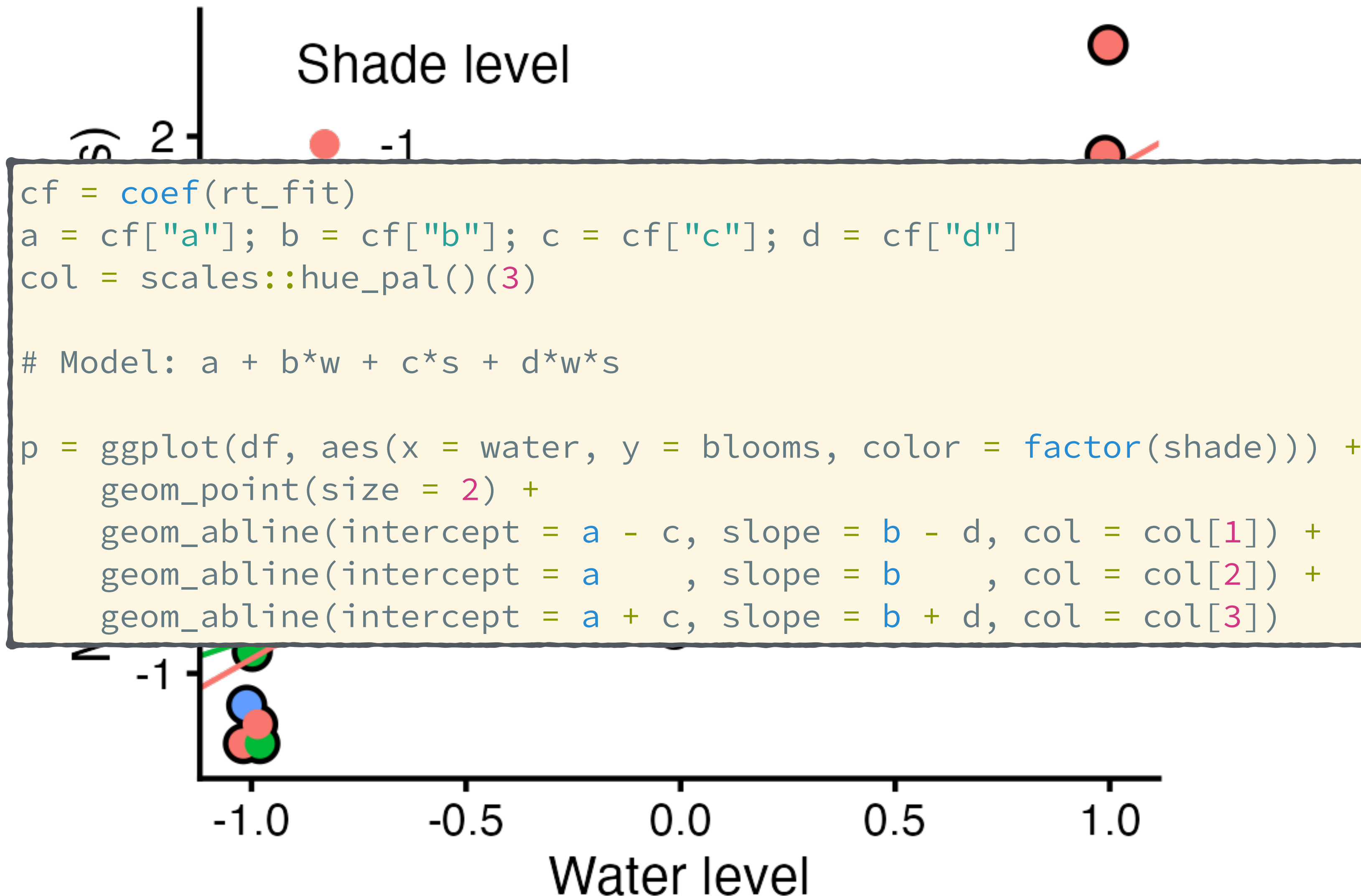
rt_fit = ulam(alist(
  blooms ~ normal(mu, sigma),
  mu <- a + b*water + c*shade + d*water*shade),
  data = df)
```



Prediction lines



Prediction lines



Summary

- Multiple linear models allow us to use more than one predictor in a linear model
 - These models do a form of automatic **stratification**
 - **Ex:** difference in size for individuals of the same age, effect of treatment for individuals of the same size
 - The objective is to compare **like-to-like**
- Coefficients can and do change with the inclusion of more predictors
 - Coefficient interpretation is hard, use plots, predictions, scaling and transformations to make models easier to interpret
 - Latter in the course, we talk about principled ways of choosing if a variable should be added to a model, stay tuned!